Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Ricerca ed analisi comparativa di sistemi di reportistica centralizzata: alternative a JasperServer

Tesi di Laurea Triennale

Relatore

Prof. Marco Zanella

Laureando

Stefano Dal Poz

Matricola 1204683

Anno Accademico 2025-2026

Stefano Dal Poz: *Ricerca ed analisi comparativa di sistemi di reportistica centralizzata: alternative a JasperServer,* Tesi di Laurea Triennale, © Aprile 2026.

Contro le difficoltà della vita le armi migliori sono il coraggio, l'ostinazione e la pazienza. Il coraggio fortifica, l'ostinazione diverte e la pazienza dà pace

— Hermann Hesse —

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata complessiva di 304 ore, dal laureando Stefano Dal Poz presso l'azienda Bluenext S.R.L. nel periodo compreso tra il 17 novembre 2025 e il 23 Gennaio 2026.
Gli obiettivi da raggiungere erano molteplici.
In primo luogo era richiesta l'analisi di un software di reportistica utilizzato presso l'azienda committente, JasperServer e Jaspersoft Studio, al fine di comprenderne le funzionalità essenziali e per definire i requisiti del nuovo sistema.
In secondo luogo era richiesto lo studio di alcune alternative, sia open source che commerciali, a JasperServer, in particolare BIRT (Business Intelligence and Reporting Tools), con lo scopo di valutarne i pro e i contro rispetto alla soluzione attualmente in uso.
Terzo ed ultimo obiettivo era lo sviluppo di un prototipo di reportistica basato su BIRT, che includesse la creazione di report semplici e complessi, la valutazione della possibilità di integrazione con un sistema di gestione lato server e la generazione di report in diversi formati (PDF, HTML, Excel, DOC).

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella, relatore della mia tesi, per l'aiuto e il sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare l'azienda Bluenext S.R.L. per avermi dato l'opportunità di svolgere il mio stage presso di loro, in particolare il mio tutor aziendale, Simone Ronchin, per la guida, la pazienza e il supporto dimostratomi durante tutto il periodo dello stage.

Desidero ringraziare con affetto i miei genitori per il sostegno, il grande aiuto e per essermi stati vicini in ogni momento durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici, in particolare Federico e Massimo, per tutte le avventure, i bei momenti vissuti e il sostegno dimostratomi durante questo cammino.

Padova, Aprile 2026 Stefano Dal Poz

Indice

1 Introduzione **1**
1.1 L'azienda . . . 1
1.1.1 Prodotti dell'azienda . . . 1
1.2 L'idea . . . 2
1.3 Organizzazione del testo . . . 2
1.3.1 Convenzioni tipografiche . . . 2

2 Descrizione dello stage **3**
2.1 Introduzione al progetto . . . 3
2.2 Analisi preventiva dei rischi . . . 3
2.3 Requisiti e obiettivi . . . 4
2.4 Pianificazione . . . 5

3 Analisi dei requisiti **6**
3.1 Tracciamento dei requisiti . . . 6
3.1.1 Requisiti funzionali . . . 7
3.1.2 Requisiti non funzionali . . . 7
3.1.3 Requisiti qualitativi . . . 7
3.1.4 Requisiti di vincolo . . . 8
3.1.5 Riepilogo dei requisiti . . . 8

4 Metodologie di analisi e tecnologie analizzate **9**
4.1 Studio del prodotto JasperServer . . . 9
4.2 Prodotti analizzati . . . 9
4.2.1 Birt . . . 9
4.2.2 Dynamic PDF . . . 10
4.2.3 FineReport . . . 10
4.2.4 Helical Insight . . . 11
4.2.5 JsReport . . . 11
4.2.6 Knowage . . . 12
4.2.7 Pentaho . . . 12
4.2.8 ReportServer . . . 13
4.2.9 SAP Crystal Report . . . 13
4.2.10 Stimulsoft . . . 14
4.3 Strumenti utilizzati . . . 14
4.4 Analisi comparativa delle tecnologie . . . 15
4.5 Analisi approfondita delle tecnologie . . . 17
4.5.1 Birt Design Report . . . 17
4.5.2 Knowage . . . 17
4.5.3 Knowage Studio . . . 18
4.5.4 Report Server . . . 19
4.5.5 Stimulsoft Designer e Stimulsoft Server . . . 19

4.6 Conclusioni 19

5 Progettazione e realizzazione del Proof of Concept 21
5.1 Obiettivi del PoC 21
5.2 Tecnologie e strumenti utilizzati 21
5.2.1 Birt Design Report 21
5.2.2 Birtruntime 22
5.2.3 Java 22
5.2.4 Python 22
5.2.5 Flask 23
5.2.6 Visual Studio Code 23
5.3 Sviluppo del programma Java 24
5.3.1 Architettura del programma 24
5.3.2 Classe BirtDesignToDocument 25
5.3.3 Attributi della classe 25
5.3.4 Metodo principale `generateDocument(String format)` 25
5.3.5 Fasi del processo di generazione del report 26
5.3.6 Considerazioni progettuali 27
5.4 Architettura del PoC 28
5.4.1 Struttura del PoC 28
5.5 Componenti principali 29
5.5.1 Motore di generazione dei report (Java / BIRT Runtime) 29
5.5.2 Server REST (Python / Flask) 30
5.5.3 Client 30
5.5.4 Vantaggi dell'architettura adottata 30
5.6 Implementazione della soluzione 31
5.6.1 Motore di generazione dei report 31
5.6.2 Implementazione del server REST 32
5.6.3 Definizione delle API REST 33
5.6.4 Client di test 34
5.6.5 Flusso di esecuzione della richiesta 35

6 Valutazione della soluzione 37
6.1 Obiettivi dei test prestazionali 37
6.2 Metodologia di test 37
6.3 Strumenti utilizzati 38
6.4 Scenari di test 38
6.5 Risultati dei test 38
6.5.1 Primo scenario: dataset ridotto 38
6.5.2 Secondo scenario: dataset medio 39
6.5.3 Terzo scenario: dataset esteso 40
6.5.4 Quarto scenario: report senza dati 40
6.6 Analisi dei risultati 40
6.7 Limiti dei test 41
6.8 Conclusioni 41

7 Conclusioni 42
7.1 Risultati ottenuti 42
7.2 Limiti del lavoro 42
7.3 Sviluppi futuri 43
7.4 Raggiungimento degli obiettivi 43
7.4.1 Requisiti funzionali 44
7.4.2 Requisiti non funzionali 44
7.4.3 Requisiti qualitativi 45

7.4.4 Requisiti di vincolo 45
7.4.5 Riepilogo dei requisiti raggiunti 45
7.5 Considerazioni finali 46
7.6 Valutazione personale 46

Acronimi e abbreviazioni **47**

Glossario **48**

Bibliografia **50**

Elenco delle figure

1.1 Logo di Bluenext S.R.L. 1

4.1 Logo di BIRT 10
4.2 Logo di Dynamic PDF 10
4.3 Logo di FineReport 11
4.4 Logo di Helical Insight 11
4.5 Logo di JsReport 12
4.6 Logo di Knowage 12
4.7 Logo di Pentaho 13
4.8 Logo di ReportServer 13
4.9 Logo di SAP Crystal Reports 14
4.10 Logo di Stimulsoft 14
4.11 Logo di Docker 15
4.12 Logo di Postman 15
4.13 Interfaccia grafica di Knowage ufficiale 18
4.14 Interfaccia grafica di Knowage installata 18

5.1 Logo di Java 22
5.2 Logo di Python 23
5.3 Logo di Flask 23
5.4 Logo di Visual Studio Code 24
5.5 Architettura del programma Java 24
5.6 Flusso di generazione del report 27
5.7 Applicativo Java per la generazione dei report 28
5.8 Architettura della soluzione proposta 31
5.9 Client di test per il server REST 35
5.10 Flusso di esecuzione del Proof of Concept 36

6.1 Logo di VisualVM 38
6.2 Tempi di generazione dei report JasperServer nel primo scenario 39
6.3 Tempi di generazione dei report PoC nel primo scenario 39
6.4 Tempi di generazione dei report JasperServer nel secondo scenario 39
6.5 Tempi di generazione dei report PoC nel secondo scenario 39
6.6 Tempi di generazione dei report JasperServer nel terzo scenario 40
6.7 Tempi di generazione dei report PoC nel terzo scenario 40
6.8 Tempi di generazione dei report JasperServer nel quarto scenario 40
6.9 Tempi di generazione dei report PoC nel quarto scenario 40

Elenco delle tabelle

2.1 Tabella degli obiettivi 5

3.1 Tabella del tracciamento dei requisti funzionali 7
3.2 Tabella del tracciamento dei requisiti non funzionali 7
3.3 Tabella del tracciamento dei requisiti qualitativi 8
3.4 Tabella del tracciamento dei requisiti di vincolo 8
3.5 Riepilogo dei requisiti 8

4.1 Confronto tra soluzioni di reportistica analizzate 16

5.1 Endpoint REST del server di reportistica 33

6.1 Tempi medi di generazione dei report nei diversi scenari 41

7.1 Tabella del tracciamento dei requisiti funzionali 44
7.2 Tabella del tracciamento dei requisiti non funzionali 44
7.3 Tabella del tracciamento dei requisiti qualitativi 45
7.4 Tabella del tracciamento dei requisiti di vincolo 45
7.5 Riepilogo dei requisiti soddisfatti 45

Capitolo 1

Introduzione

In questo capitolo viene presentata l'azienda ospitante, l'idea alla base dello stage e l'organizzazione del testo.

1.1 L'azienda

Il progetto di *stage* di questa tesi è stato svolto presso l'azienda Bluenext Srl, società di consulenza informatica fondata nel 2012 a Rimini e operativa su tutto il territorio nazionale. In particolare, lo *stage* si è svolto presso la sede di Treviso, in una delle filiali distribuite sul territorio italiano, che in passato operava con il nome di SogeaSoft Srl prima di essere acquisita da Bluenext nel 2022.

Figura 1.1: Logo di Bluenext S.R.L.

1.1.1 Prodotti dell'azienda

L'azienda offre soluzioni *software* personalizzate per imprese di varie dimensioni, con l'obiettivo di migliorare i processi aziendali e aumentare l'efficienza operativa. Il target principale è rappresentato dalle Piccole e Medie Imprese (PMI) che desiderano digitalizzare i propri processi e richiedono supporto tecnico specializzato. L'azienda si occupa dello sviluppo di *software* ERP e il suo prodotto di punta è «SAI», un *software* gestionale caratterizzato da una elevata flessibilità che permette di effettuare integrazioni in qualsiasi direzione, rendendolo estremamente adattabile e personalizzabile in base alle specifiche esigenze del cliente.

1.2 L'idea

I sistemi di reportistica centralizzata sono strumenti essenziali per le aziende che necessitano di generare, gestire e distribuire report in modo efficiente. JasperServer è una soluzione ampiamente adottata, ma esistono alternative che potrebbero offrire vantaggi in termini di prestazioni, scalabilità, costi e facilità di integrazione. Questo progetto mira a valutare diverse alternative a JasperServer, analizzandone le caratteristiche, i punti di forza e le potenziali limitazioni arrivando a sviluppare un prototipo basato su una delle soluzioni individuate, al fine di testarne l'efficacia in un contesto reale.

1.3 Organizzazione del testo

Il capitolo corrente è l'introduzione del documento, dove si è spiegato brevemente l'ambito di lavoro e il progetto sul quale si è svolto lo *stage*. In seguito il documento sarà organizzato con la seguente struttura:

Il secondo capitolo approfondisce la descrizione dello stage, illustrando il contesto aziendale, le tecnologie utilizzate e le sfide affrontate durante il periodo di tirocinio.

Il terzo capitolo approfondisce l'analisi dei requisiti del sistema di reportistica, descrivendo il tracciamento dei requisiti.

Il quarto capitolo approfondisce il progetto di *stage*, descrivendo gli obiettivi e le attività svolte, la metodologia di lavoro adottata e le tecnologie analizzate.

Il quinto capitolo è dedicato all'analisi delle soluzioni individuate, alla progettazione e alla codifica del prototipo di reportistica basato su una delle alternative a JasperServer individuate.

Il sesto capitolo descrive le prestazioni ottenute durante i test effettuati sul prototipo di reportistica sviluppato rispetto a JasperServer.

Nel settimo capitolo descrive le conclusioni del lavoro svolto, riassumendo i risultati ottenuti e proponendo possibili sviluppi futuri.

1.3.1 Convenzioni tipografiche

Riguardo la stesura del testo, relativamente al documento sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato alla fine del presente documento;
- per la prima occorrenza dei termini riportati nel glossario viene utilizzata la seguente nomenclatura: *parola*[g];
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*;
- i comandi, le query, i percorsi di file e il codice inline sono rappresentati tramite il carattere `monospaziato`.

Capitolo 2

Descrizione dello stage

In questo capitolo viene presentata la descrizione dello stage*, con particolare attenzione al progetto svolto, ai rischi identificati e agli obiettivi raggiunti.*

2.1 Introduzione al progetto

Lo *stage* si è svolto presso l'azienda Bluenext Srl, specializzata nello sviluppo di soluzioni software personalizzate per le imprese. L'obiettivo principale dello *stage* è stato quello di ricercare e analizzare sistemi di reportistica centralizzata, con particolare attenzione alle alternative a JasperServer. Il progetto ha previsto una fase di analisi delle soluzioni esistenti, seguita dalla progettazione e implementazione di un prototipo basato su una delle alternative individuate.

2.2 Analisi preventiva dei rischi

L'analisi preventiva dei rischi ha l'obiettivo di identificare possibili criticità tecniche, organizzative e prestazionali, al fine di definire strategie di mitigazione e garantire l'affidabilità della soluzione proposta.
In una prima fase di analisi iniziale sono stati individuati alcuni possibili rischi a cui si potrà andare incontro. Si è quindi proceduto a elaborare delle possibili soluzioni per far fronte a tali rischi.

1. Non conoscenza di *JasperServer*

Descrizione: La mancata conoscenza iniziale di *JasperServer* potrebbe ostacolare la valutazione e la ricerca delle alternative.

Impatto: Alto.

Probabilità: Media.

Soluzione: Per mitigare questo rischio, è stato pianificato uno studio approfondito della documentazione ufficiale di *JasperServer* e l'analisi di casi d'uso reali per comprendere le funzionalità chiave e i requisiti che le alternative devono soddisfare.

2. Integrazione tra componenti eterogenei

Descrizione: L'integrazione tra componenti sviluppati in ambienti e linguaggi differenti (*Java*[g] per il motore di reportistica e *Python*[g] per il server *REST*[g]) potrebbe generare problemi di comunicazione, gestione degli errori o incompatibilità nei formati dei dati scambiati.

Impatto: Alto.

Probabilità: Media.

Soluzione: Definizione di interfacce *REST* standardizzate, utilizzo di formati di scambio dati strutturati (*JSON*[g]) e *test* incrementali delle *API*[g] per verificare il corretto funzionamento delle interazioni tra i componenti.

3. Complessità di installazione e configurazione

Descrizione: La complessità di installazione e configurazione delle soluzioni alternative a *JasperServer* potrebbe richiedere tempi e risorse maggiori rispetto alle aspettative iniziali.

Impatto: Medio.

Probabilità: Media.

Soluzione: Redazione di una documentazione tecnica dettagliata delle procedure di installazione e configurazione e utilizzo di container *Docker* per standardizzare l'ambiente di esecuzione.

4. Scalabilità del sistema

Descrizione: La scalabilità del sistema potrebbe essere limitata da fattori tecnologici o architetturali, specialmente in caso di crescita del carico di lavoro o dell'uso simultaneo da parte di molti utenti.

Impatto: Alto.

Probabilità: Media.

Soluzione: Valutazione della scalabilità attraverso *test* di carico e analisi delle architetture supportate. Eventuale separazione tra componente di generazione dei report e componente di esposizione dei risultati.

5. Dipendenza da tecnologie di terze parti

Descrizione: L'utilizzo di *framework* e librerie di terze parti (*JasperReports*, *BIRT Runtime*, *Java*, *Python*) introduce una dipendenza da tecnologie esterne, con possibili problemi di compatibilità, supporto o aggiornamenti futuri.

Impatto: Medio.

Probabilità: Bassa.

Soluzione: Selezione di versioni stabili e ben documentate, verifica della compatibilità tra le dipendenze e definizione di una strategia di aggiornamento controllata.

2.3 Requisiti e obiettivi

Gli obiettivi principali dello *stage* sono stati definiti in base alle esigenze dell'azienda ospitante e alle competenze da sviluppare durante il periodo di tirocinio.
Le fasi specifiche del progetto sono state le seguenti:

- Ricerca e analisi di alternative a *JasperServer* per la reportistica centralizzata.
- Progettazione e implementazione di un prototipo basato su una delle soluzioni individuate.
- Valutazione delle prestazioni del prototipo sviluppato.
- Documentazione dettagliata del progetto e preparazione della presentazione finale.

Ogni fase è stata pianificata con obiettivi specifici e tempistiche definite per garantire il completamento del progetto entro la durata dello *stage*.
Nella tabella 2.1 sono elencati gli obiettivi dello *stage*.

Tabella 2.1: Tabella degli obiettivi

Obiettivi Obbligatori	
OB1	Studio documentazione e analisi comparativa
OB2	Valutazione e implementazione *PoC*[g] di due soluzioni alternative
OB3	Documentazione configurazione
Obiettivi Desiderabili	
DE1	Analisi costi e licenze
DE2	Raccolta feedback con tutor o key user da lui individuati
DE3	Definizione best practice di integrazione
Obiettivi Facoltativi	
FA1	Valutazione altre alternative
FA2	Integrazione di funzioni di reporting personalizzato

Per l'analisi dei requisiti si rimanda al Capitolo 3 per definirli in dettaglio e in modo più strutturato.

2.4 Pianificazione

Tutto lo svolgimento dello *stage* è stato pianificato in modo da rispettare le tempistiche stabilite e garantire il raggiungimento degli obiettivi prefissati seguendo la metodologia *Agile*. Le attività previste per lo svolgimento dello *stage* sono state suddivise in cinque fasi principali:

- **Fase 1 - Analisi delle soluzioni esistenti:** ricerca e valutazione delle alternative a JasperServer per la reportistica centralizzata.
- **Fase 2 - Analisi approfondita delle soluzioni individuate:** studio dettagliato delle funzionalità, dei vantaggi e degli svantaggi delle soluzioni selezionate.
- **Fase 3 - Studio specifiche e implementazione del prototipo:** definizione delle specifiche tecniche e sviluppo del prototipo basato sulla soluzione scelta.
- **Fase 4 - Test e valutazione:** esecuzione di *test* per verificare le funzionalità e le performance del prototipo, seguita da una valutazione complessiva.
- **Fase 5 - Documentazione e presentazione:** redazione della documentazione e preparazione della presentazione finale del progetto.

Capitolo 3

Analisi dei requisiti

Questo capitolo presenta l'analisi dettagliata dei requisiti funzionali, qualitativi e di vincolo del progetto.

3.1 Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e degli obiettivi effettuata sul progetto è stata stilata la tabella che traccia i requisiti. Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.
Ogni requisito analizzato sarà identificato univocamente da una sigla del tipo **R[Tipo][Priorità][Codice]** nella quale:

R sta per Requisito;

[Tipo] può essere:

- [**F**] se Funzionale;
- [**NF**] se Non Funzionale;
- [**Q**] se di Qualità;
- [**V**] se di Vincolo.

[Priorità] può essere:

- [**O**] per Obbligatorio;
- [**D**] per Desiderabile;
- [**P**] per Opzionale.

[Codice] : identifica univocamente i requisiti per ogni tipologia. È un numero intero progressivo univoco assegnato in ordine di importanza se il requisito non ha padre, se invece si tratta di un sotto-requisito segue il formato **[Codice padre].[Numero figlio]** e trattandosi di una struttura ricorsiva non c'è limite alla profondità della gerarchia. I codici sono numerati in base alla sezione e alla classificazione (Es: RFO1 = Requisito Funzionale Obbligatorio 1, RNFO1 = Requisito Non Funzionale Obbligatorio 1, RQO1 = Requisito di Qualità Obbligatorio 1, RVO1 = Requisito di Vincolo Obbligatorio 1).

Nelle tabelle 3.1, 3.2, 3.3 e 3.4 sono riassunti i requisiti emersi in fase di analisi, classificati in base alla loro priorità e accompagnati da una breve descrizione della relativa funzionalità.

3.1.1 Requisiti funzionali

I requisiti funzionali descrivono cosa deve fare il sistema. Sono le funzionalità concrete che la soluzione deve offrire per raggiungere gli obiettivi del progetto.

Tabella 3.1: Tabella del tracciamento dei requisti funzionali

Requisito	Descrizione	Classificazione
RFO1	Il sistema deve fornire un editor grafico per consentire la creazione e la progettazione visuale dei report	Obbligatorio
RFO2	Il sistema deve permettere l'integrazione con servizi e applicazioni che espongono dati tramite *API* in formato *JSON*	Obbligatorio
RFO3	Il sistema deve poter gestire report complessi e strutturati in modo modulare, in particolare elementi di sottoreport	Obbligatorio
RFO4	Il sistema deve consentire l'esportazione dei report in molteplici formati (*PDF*[g], DOC, XLSX)	Obbligatorio
RFO5	Il sistema deve consentire l'invocazione programmatica dei report da applicazioni esterne tramite *API*	Obbligatorio
RFO6	Il sistema deve garantire supporto a database *SQL*[g] e *NoSQL*	Obbligatorio
RFO7	Il sistema deve fornire la possibilità di personalizzazione e *scripting* dei report	Obbligatorio

3.1.2 Requisiti non funzionali

I requisiti non funzionali definiscono come il sistema deve comportarsi, cioè le sue proprietà di qualità interna. Non aggiungono nuove funzioni, ma impongono vincoli di prestazioni, sicurezza, disponibilità, scalabilità, affidabilità e manutenibilità.

Tabella 3.2: Tabella del tracciamento dei requisiti non funzionali

Requisito	Descrizione	Classificazione
RNFO1	Il sistema deve supportare la scalabilità, consentendo la gestione di un numero crescente di richieste e utenti	Obbligatorio
RNFO2	Il sistema deve garantire la corretta generazione dei report in modo consistente	Obbligatorio
RNFO3	Il sistema deve consentire una facile integrazione con applicazioni esistenti sviluppate in *Java*, *.NET* e *Python*	Obbligatorio
RNFO4	Il sistema deve garantire tempi di risposta accettabili anche su dataset consistenti	Obbligatorio
RNFO5	Il sistema deve risultare mantenibile, facilitando aggiornamenti, configurazioni e interventi evolutivi	Desiderabile

3.1.3 Requisiti qualitativi

I requisiti qualitativi specificano le proprietà qualitative che influenzano l'esperienza d'uso e la manutenibilità. Si concentrano su aspetti percepibili, come semplicità, chiarezza, flessibilità o estendibilità.

Tabella 3.3: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Classificazione
RQO1	Il sistema deve fornire un'interfaccia utente intuitiva e di facile utilizzo	Obbligatorio
RQD2	Il sistema deve garantire coerenza nella progettazione e nel *layout* dei report	Desiderabile
RQD3	Il sistema deve offrire una buona esperienza di sviluppo, supportata da documentazione chiara e *API* ben definite	Desiderabile

3.1.4 Requisiti di vincolo

I requisiti di vincolo impongono limitazioni o condizioni esterne al progetto: ambienti, tecnologie, compatibilità, strumenti, standard aziendali o legali.

Tabella 3.4: Tabella del tracciamento dei requisiti di vincolo

Requisito	Descrizione	Classificazione
RVO1	Il sistema deve poter essere installato e gestito su server locali, senza dipendere esclusivamente da soluzioni cloud	Obbligatorio
RVO2	Il sistema deve prevedere condizioni di *licensing* sostenibili, evitando vincoli restrittivi legati a licenze proprietarie	Obbligatorio
RVO3	Il sistema deve essere compatibile con l'infrastruttura e lo *stack* tecnologico aziendale esistente	Desiderabile
RVD4	Il sistema dovrebbe utilizzare tecnologie consolidate e ampiamente supportate dalla *community* o dal *vendor*	Desiderabile

3.1.5 Riepilogo dei requisiti

Nella tabella 3.5 è riportato un riepilogo dei requisiti individuati, suddivisi per tipologia e classificazione. Questo consente di avere una panoramica chiara e sintetica dell'insieme dei requisiti che guideranno lo sviluppo del progetto.

Tabella 3.5: Riepilogo dei requisiti

Tipologia	Obbligatorio	Desiderabile	Opzionale	Totale
Funzionali	7	0	0	7
Non funzionali	4	1	0	5
Qualitativi	1	2	0	3
Vincolo	3	1	0	4
Totale	15	4	0	19

Capitolo 4

Metodologie di analisi e tecnologie analizzate

Questo capitolo descrive le diverse fasi di ricerca ed analisi utilizzate per analizzare un prodotto a partire dallo studio di JasperServer mettendo in luce le diverse tecnologie considerate

4.1 Studio del prodotto JasperServer

La prima fase del progetto ha previsto uno studio approfondito del prodotto JasperServer[1], al fine di comprenderne le funzionalità, le caratteristiche tecniche e le modalità di utilizzo per poter . Per analizzare il prodotto JasperServer sono state utilizzate diverse metodologie di analisi, tra cui:

- **Analisi documentale**: è stata effettuata una revisione approfondita della documentazione ufficiale di JasperServer, inclusi manuali utente, guide di installazione e documenti tecnici. Questo ha permesso di comprendere le funzionalità principali del prodotto, le sue caratteristiche tecniche e le modalità di utilizzo.
- **Test pratici**: sono stati eseguiti test pratici utilizzando JasperServer in scenari reali o simulati. Questo ha permesso di valutare le prestazioni del prodotto, la facilità di configurazione e l'efficacia delle funzionalità offerte.

L'analisi di JasperServer ha fornito una base solida per la successiva fase di ricerca delle alternative, consentendo di identificare i requisiti chiave che le soluzioni alternative devono soddisfare, presenti alla tabella 3.1.

4.2 Prodotti analizzati

Durante il progetto sono state analizzate diverse tecnologie con lo scopo di individuare quale di queste fosse la più adatta a sostituire JasperServer. Per eseguire questa ricerca sono stati consultati articoli, documentazioni ufficiali e tutorial online.

4.2.1 Birt

Birt[2] (Business Intelligence and Reporting Tools) è una piattaforma open source per la creazione di report e visualizzazioni di dati. È sviluppata dalla Eclipse Foundation ed è

[1] *JasperReports Library*. URL: https://community.jaspersoft.com/project/jasperreports-library.
[2] *BIRT - Business Intelligence and Reporting Tools*. URL: https://eclipse-birt.github.io/birt-website/.

ampiamente utilizzata per lo sviluppo di applicazioni di business intelligence. Birt offre una vasta gamma di funzionalità, tra cui la creazione di report interattivi, la generazione di grafici e tabelle, l'integrazione con diverse fonti di dati e la possibilità di personalizzare l'aspetto dei report. La piattaforma Birt è basata su *Java* e può essere integrata in applicazioni web o desktop. Offre un ambiente di sviluppo intuitivo che consente agli sviluppatori di creare report complessi senza dover scrivere codice complesso. Birt supporta anche l'esportazione dei report in diversi formati, come *PDF*, Excel, *HTML*[g] e Word, consentendo agli utenti di condividere facilmente i report generati. In sintesi, Birt è una piattaforma potente e flessibile per la creazione di report e visualizzazioni di dati, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle applicazioni di business intelligence.

Figura 4.1: Logo di BIRT

4.2.2 Dynamic PDF

Dynamic PDF[3]è una libreria software che consente la generazione dinamica di documenti *PDF* in modo programmatico. È sviluppata da Ceetron Solutions ed è ampiamente utilizzata per creare report, fatture, etichette e altri documenti *PDF* personalizzati. La libreria Dynamic *PDF* offre una vasta gamma di funzionalità, tra cui la creazione di pagine, l'aggiunta di testo, immagini, tabelle e grafici, nonché la formattazione avanzata dei contenuti. Supporta anche l'inserimento di collegamenti ipertestuali, segnalibri e moduli interattivi nei documenti *PDF*. Dynamic *PDF* è progettata per essere facile da usare e integrare in applicazioni software esistenti. Offre un'*API* intuitiva che consente agli sviluppatori di creare documenti *PDF* in modo rapido ed efficiente, senza dover affrontare la complessità del formato *PDF* sottostante. In sintesi, Dynamic *PDF* è una libreria potente e flessibile per la generazione dinamica di documenti *PDF*, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle applicazioni che richiedono la creazione di documenti *PDF* personalizzati.

Figura 4.2: Logo di Dynamic PDF

4.2.3 FineReport

FineReport[4] è una piattaforma di business intelligence e reportistica sviluppata da FanRuan Software Co., Ltd. È ampiamente utilizzata per la creazione di report, dashboard e analisi

[3] *DynamicPDF*. URL: https://www.dynamicpdf.com/.
[4] *FineReport*. URL: https://www.finereport.com/.

dei dati in ambito aziendale. FineReport offre una vasta gamma di funzionalità, tra cui la creazione di report interattivi, la generazione di grafici e tabelle, l'integrazione con diverse fonti di dati e la possibilità di personalizzare l'aspetto dei report. La piattaforma supporta anche l'esportazione dei report in diversi formati, come *PDF*, Excel, *HTML* e Word. FineReport è progettata per essere facile da usare e integrare in applicazioni software esistenti. Offre un ambiente di sviluppo intuitivo che consente agli sviluppatori di creare report complessi senza dover scrivere codice complesso. In sintesi, FineReport è una piattaforma potente e flessibile per la creazione di report e analisi dei dati, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle applicazioni di business intelligence.

Figura 4.3: Logo di FineReport

4.2.4 Helical Insight

Helical Insight[5] è una piattaforma di business intelligence open source che offre una vasta gamma di strumenti per l'analisi dei dati, la creazione di report e la visualizzazione delle informazioni aziendali. È progettata per essere flessibile e scalabile, consentendo alle organizzazioni di adattare la piattaforma alle proprie esigenze specifiche. Helical Insight offre funzionalità avanzate come l'integrazione con diverse fonti di dati, la creazione di dashboard interattivi, l'analisi predittiva e l'elaborazione in-memory. La piattaforma supporta anche l'esportazione dei report in diversi formati, come *PDF*, Excel, *HTML* e Word. Helical Insight è sviluppata in *Java* e può essere integrata in applicazioni web o desktop. Offre un ambiente di sviluppo intuitivo che consente agli sviluppatori di creare report complessi senza dover scrivere codice complesso. In sintesi, Helical Insight è una piattaforma completa di business intelligence che consente alle organizzazioni di sfruttare al massimo i propri dati per prendere decisioni informate e migliorare le prestazioni aziendali.

Figura 4.4: Logo di Helical Insight

4.2.5 JsReport

JsReport[6] è una piattaforma di reportistica open source che consente la creazione, la gestione e la distribuzione di report in modo semplice ed efficiente. È progettata per essere flessibile e scalabile, consentendo agli sviluppatori di creare report personalizzati utilizzando tecnologie web standard come *HTML*, CSS e JavaScript. JsReport offre una vasta gamma di funzionalità, tra cui la generazione di report in diversi formati (*PDF*, Excel, HTML), l'integrazione con

[5] *Helical Insight*. URL: https://www.helicalinsight.com/.
[6] *JSPReport*. URL: https://www.jsreport.net/.

diverse fonti di dati, la creazione di dashboard interattivi e la possibilità di programmare l'esecuzione dei report. La piattaforma supporta anche l'uso di modelli personalizzati e l'estensione delle funzionalità tramite plugin. JsReport è sviluppata in Node.js e può essere integrata in applicazioni web o desktop. Offre un'*API* intuitiva che consente agli sviluppatori di creare report complessi senza dover scrivere codice complesso. In sintesi, JsReport è una piattaforma potente e flessibile per la creazione di report, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle applicazioni di reportistica.

Figura 4.5: Logo di JsReport

4.2.6 Knowage

Knowage[7] è una piattaforma di business intelligence open source che offre una vasta gamma di strumenti per l'analisi dei dati, la creazione di report e la visualizzazione delle informazioni aziendali. È sviluppata da Engineering Ingegneria Informatica S.p.A. ed è ampiamente utilizzata da organizzazioni di diverse dimensioni per prendere decisioni basate sui dati. Knowage offre una suite completa di strumenti, tra cui Knowage Reporting per la creazione di report, Knowage OLAP per l'analisi multidimensionale dei dati e Knowage Dashboard per la creazione di dashboard interattivi. La piattaforma supporta l'integrazione con diverse fonti di dati, inclusi database relazionali, file di testo, servizi web e big data. Knowage offre anche funzionalità avanzate come l'analisi predittiva, l'elaborazione in-memory e l'integrazione con strumenti di visualizzazione dei dati come Tableau e QlikView. In sintesi, Knowage è una piattaforma completa di business intelligence che consente alle organizzazioni di sfruttare al massimo i propri dati per prendere decisioni informate e migliorare le prestazioni aziendali.

Figura 4.6: Logo di Knowage

4.2.7 Pentaho

Pentaho[8] è una piattaforma di business intelligence open source che offre una vasta gamma di strumenti per l'analisi dei dati, la creazione di report e la visualizzazione delle informazioni aziendali. È sviluppata da Hitachi Vantara ed è ampiamente utilizzata da organizzazioni di diverse dimensioni per prendere decisioni basate sui dati. Pentaho offre una suite completa di strumenti, tra cui Pentaho Data Integration (PDI) per l'estrazione, la trasformazione e il caricamento dei dati (ETL), Pentaho Business Analytics per l'analisi dei dati e la creazione

[7] *Knowage*. URL: https://www.knowage-suite.com/.
[8] *Pentaho*. URL: https://pentaho.com/.

di report, e Pentaho Dashboard Designer per la creazione di dashboard interattivi. La piattaforma supporta l'integrazione con diverse fonti di dati, inclusi database relazionali, file di testo, servizi web e big data. Offre anche funzionalità avanzate come l'analisi predittiva, l'elaborazione in-memory e l'integrazione con strumenti di visualizzazione dei dati come Tableau e QlikView. In sintesi, Pentaho è una piattaforma completa di business intelligence che consente alle organizzazioni di sfruttare al massimo i propri dati per prendere decisioni informate e migliorare le prestazioni aziendali.

Figura 4.7: Logo di Pentaho

4.2.8 ReportServer

ReportServer[9] è una piattaforma di reportistica open source che consente la creazione, la gestione e la distribuzione di report in modo semplice ed efficiente. È progettata per essere flessibile e scalabile, consentendo agli sviluppatori di creare report personalizzati utilizzando tecnologie web standard come *HTML*, CSS e JavaScript. ReportServer offre una vasta gamma di funzionalità, tra cui la generazione di report in diversi formati (*PDF*, Excel, HTML), l'integrazione con diverse fonti di dati, la creazione di dashboard interattivi e la possibilità di programmare l'esecuzione dei report. La piattaforma supporta anche l'uso di modelli personalizzati e l'estensione delle funzionalità tramite plugin. ReportServer è sviluppata in *Java* e può essere integrata in applicazioni web o desktop. Offre un'*API* intuitiva che consente agli sviluppatori di creare report complessi senza dover scrivere codice complesso. In sintesi, ReportServer è una piattaforma potente e flessibile per la creazione di report, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle applicazioni di reportistica.

Figura 4.8: Logo di ReportServer

4.2.9 SAP Crystal Report

SAP Crystal Reports[10] è uno strumento di reportistica sviluppato da SAP che consente agli utenti di creare report interattivi e visivamente accattivanti basati su dati provenienti da diverse fonti. È ampiamente utilizzato in ambito aziendale per generare report dettagliati e analisi dei dati. SAP Crystal Reports offre una vasta gamma di funzionalità, tra cui la

[9] *Report Server*. URL: https://reportserver.net/en/.
[10] *SAP Crystal Reports*. URL: https://www.sap.com/products/crystal-reports.html.

creazione di report personalizzati, l'integrazione con diverse fonti di dati (database relazionali, file di testo, servizi web), la generazione di grafici e tabelle, e la possibilità di esportare i report in diversi formati (*PDF*, Excel, Word). Lo strumento supporta anche funzionalità avanzate come la formattazione condizionale, i sottoreport, le formule personalizzate e l'integrazione con applicazioni .NET e Java. In sintesi, SAP Crystal Reports è uno strumento potente e flessibile per la creazione di report aziendali, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle organizzazioni che necessitano di analisi dei dati e reportistica avanzata.

Figura 4.9: Logo di SAP Crystal Reports

4.2.10 Stimulsoft

Stimulsoft[11] è una suite di strumenti di reportistica e business intelligence che consente agli utenti di creare report interattivi e dashboard personalizzati. È progettata per essere facile da usare e altamente flessibile, supportando diverse fonti di dati e offrendo un'ampia gamma di funzionalità per la generazione e la visualizzazione dei dati. Stimulsoft offre un ambiente visivo intuitivo per la creazione di report, con supporto per diversi formati di output come *PDF*, Excel, Word e *HTML*. Inoltre, consente l'integrazione con applicazioni web e desktop, fornendo *API* e librerie che facilitano l'automazione della generazione dei report. In sintesi, Stimulsoft è uno strumento potente e versatile per la creazione di report aziendali, che offre una vasta gamma di funzionalità per soddisfare le esigenze delle organizzazioni che necessitano di analisi dei dati e reportistica avanzata.

Figura 4.10: Logo di Stimulsoft

4.3 Strumenti utilizzati

Per lo studio dei vari programmi di reportistica sono stati utilizzati i seguenti strumenti:

- ***Docker e Docker Compose***:[12]una piattaforma di containerizzazione che consente di creare, distribuire e eseguire applicazioni in ambienti isolati chiamati container. Docker semplifica la gestione delle dipendenze e garantisce la coerenza tra gli ambienti di sviluppo, test e produzione. Docker Compose è uno strumento che consente di definire e gestire applicazioni multi-container utilizzando un file di configurazione `docker-compose.yml`. In particolare, Docker è stato utilizzato per creare un ambiente di sviluppo isolato per le diverse applicazioni, consentendo di eseguire le applicazioni in

[11] *Stimulsoft*. URL: https://www.stimulsoft.com/en/.
[12] *Docker*. URL: https://www.docker.com/.

un container separato dal sistema operativo host. Docker Compose è stato utilizzato per definire e gestire i servizi necessari per l'esecuzione degli applicativi, come il database e il server applicativo, dove possibile.

Figura 4.11: Logo di Docker

- ***Postman***:[13]uno strumento utilizzato per testare le *API RESTful*. Consente di inviare richieste *HTML*[g] e visualizzare le risposte, facilitando il processo di sviluppo e debug delle *API*. E' stato utilizzato per creare un server mock sul quale eseguire test sulle *API* messe a disposizione dai vari software di reportistica e simulare passaggi di dati in formato *JSON*.

Figura 4.12: Logo di Postman

4.4 Analisi comparativa delle tecnologie

La seguente fase del progetto ha previsto un'analisi comparativa delle tecnologie individuate nella sezione 4.2, al fine di valutare le loro caratteristiche, funzionalità e prestazioni rispetto ai requisiti identificati nello studio di JasperServer, definiti nella tabella 3.1.
Per ciascuna piattaforma sono state analizzate le informazioni disponibili nella documentazione ufficiale, includendo guide di installazione, manuali utente, riferimenti *API* ed esempi di utilizzo. Questa fase ha consentito di acquisire una visione preliminare delle funzionalità offerte, dell'architettura del sistema e delle modalità di integrazione con applicazioni esterne.
Particolare attenzione è stata posta alla disponibilità di un editor grafico per la progettazione dei report, al supporto per sorgenti dati eterogenee (database *SQL* e NoSQL, formati *JSON*) e alle modalità di generazione dei report tramite *API*. Sono stati inoltre valutati gli aspetti relativi al deploy della piattaforma, verificando la possibilità di installazione su server locali e l'eventuale dipendenza da soluzioni cloud.
Lo studio della documentazione ha incluso anche una valutazione preliminare dell'esperienza di sviluppo, analizzando la chiarezza della documentazione, la presenza di esempi e la maturità delle *API* fornite. Infine, sono stati considerati gli aspetti di licensing, al fine di individuare eventuali limitazioni o vincoli incompatibili con il contesto aziendale.
Sulla base delle informazioni raccolte, è stato possibile effettuare una prima selezione delle soluzioni più promettenti, individuando quelle che soddisfacevano i requisiti obbligatori e che risultavano maggiormente adatte a essere approfondite attraverso la realizzazione di un Proof of Concept.
Lo studio della documentazione ufficiale ha permesso di ridurre il rischio di implementazione, orientando la scelta verso soluzioni coerenti con i requisiti aziendali prima di procedere alla fase di sperimentazione pratica.
Di seguito, nella tabella 4.1, viene riportata una comparazione delle tecnologie analizzate, evidenziando la tipologia, i principali motivi di inclusione e i limiti principali rispetto ai requisiti individuati:

[13] *Postman - API Development Environment*. URL: https://www.postman.com/.

Soluzione	Tipologia	Motivo di inclusione	Limiti principali
BIRT + Knowage	Open-source	Combina editor grafico e server di gestione report; copre gran parte dei requisiti funzionali	Sottoreport non supportati in modo chiaro; supporto *JSON* solo tramite script manuali; upload report non automatizzato
DynamicPDF	Commerciale	Buon supporto alla generazione *PDF* e ai sottoreport	Output limitato al PDF; assenza di *API* di generazione; supporto *JSON* limitato
FineReport	Commerciale	Editor grafico avanzato; server locale; diversi formati di output	Mancanza di *API*; *JSON* solo locale e tramite scripting; assenza di sottoreport
Helical Insight	Commerciale	Soluzione completa lato report e server; buona facilità d'uso	Versione open-source non più supportata; assenza di *API*; sottoreport non disponibili
JasperReport	Open-source	Buon supporto *API*; *JSON* nativo; server locale; diversi formati di output	Assenza di editor grafico visuale; report da definire in *HTML*; sottoreport non supportati
Pentaho Report Designer + Server	Commerciale	Ambiente di design maturo; supporto sottoreport; server locale	*JSON* solo tramite file locale; assenza di *API* di generazione
Report Server	Open-source e Commerciale	Piattaforma di gestione centralizzata dei report; supporto a diversi formati	Non include editor grafico; copre solo la parte server
Stimulsoft Designer + Server	Commerciale	Piattaforma quasi completa; editor grafico avanzato; supporto *JSON* e sottoreport	*API* non native; installazione server limitata (Windows / Docker)
SAP Crystal Report	Commerciale	Editor grafico completo; supporto sottoreport; ampia diffusione	Solo ambiente Windows; *JSON* non nativo; non include server di gestione

Tabella 4.1: Confronto tra soluzioni di reportistica analizzate

Sulla base di questa analisi preliminare, alcune soluzioni sono state escluse in quanto non soddisfacevano i requisiti obbligatori definiti, mentre altre sono state ritenute idonee per un'analisi più approfondita.
In particolare le soluzioni escluse sono state le seguenti: DynamicPDF, FineReport, Helical Insight, Jsreport, Pentaho Report Designer + Server, SAP Crystal report. Quelle sottoposte a un'analisi più approfondita sono Birt + Knowage, Report Server e Stimulsoft Designer + Server in quanto rispettano un maggior numero di requisiti obbligatori.

4.5 Analisi approfondita delle tecnologie

La fase successiva del progetto ha previsto un'analisi approfondita delle soluzioni selezionate, al fine di valutare in modo più dettagliato le loro funzionalità, prestazioni e facilità d'uso.

4.5.1 Birt Design Report

Il prodotto Birt Design Report è un editor per report realizzato sulla piattaforma Eclipse che permette di realizzare report tramite un editor grafico in modo similare a quanto offerto da JasperStudio ed è stato considerato per la parte di realizzazione di report in locale tramite editor grafico. Viene offerto in forma open-source attualmente in versione 4.22.0 ma per l'analisi è stata considerata la versione 4.21.0 in quanto era quella disponibile in quel momento.
L'analisi nella sua prima fase si è concentrata nel verificare la presenza e la possibilità di utilizzare dati passati in formati *JSON* e forniti tramite parametro per popolare il report. Da questa analisi è emerso che per poter replicare questa funzionalità non presente nativamente è necessario definire uno script in Javascript che dato in input un parametro di tipo stringa lo utilizzi per scaricare i dati in formato *JSON* successivamente utilizzabili per riempire il report. Per realizzare questo si è proceduto in questo modo:

- Definire un data source di tipo Scripted Data Source;
- Aggiungere un parametro, che chiameremo json, in Report Parameters;
- Creare un dataset collegandolo al datasource precedentemente definito;
- Sul dataset appena creato, nel file di design seleziono `Script` e successivamente in alto seleziono `open` dove aggiungo il seguente codice:

4.5.2 Knowage

Questo prodotto è stato considerato per la parte server per replicare le funzionalità di JasperServer in quanto mette a disposizione un'implementazione grafica per ospitare report realizzati con Birt. Per poterlo analizzare è stato scelto di utilizzare un'immagine *Docker*[g] ufficiale che fornisce il prodotto in versione 9.0, in quanto è una soluzione che permette di eseguire l'applicazione in un ambiente isolato e facilmente replicabile, e anche l'installazione locale, che fornisce il prodotto in versione 8.1.7 in modo da poter verificare nel modo più completo possibile l'effettiva disponibilità delle funzionalità offerte.
L'analisi in questo punto è stata focalizzata principalmente sulla possibilità di eseguire il deploy di report da programma Birt Design Report, sulla possibilità di eseguire i report tramite *API* e sulla possibilità di passare dati in formato *JSON* tramite parametro.
Le prove per poter eseguire il deploy dei report da Birt Design Report sono state effettuate sia tramite l'interfaccia grafica di Knowage che tramite *API*.
Per quanto riguarda l'interfaccia grafica essa presenta alcune discrepanze rispetto alla documentazione ufficiale, come visibile nelle figure 4.13 e 4.14, in quanto non è presente una

sezione dedicata al deploy dei report ne al caricamento degli stessi, come è possibile verificare nelle immagini sottostanti.

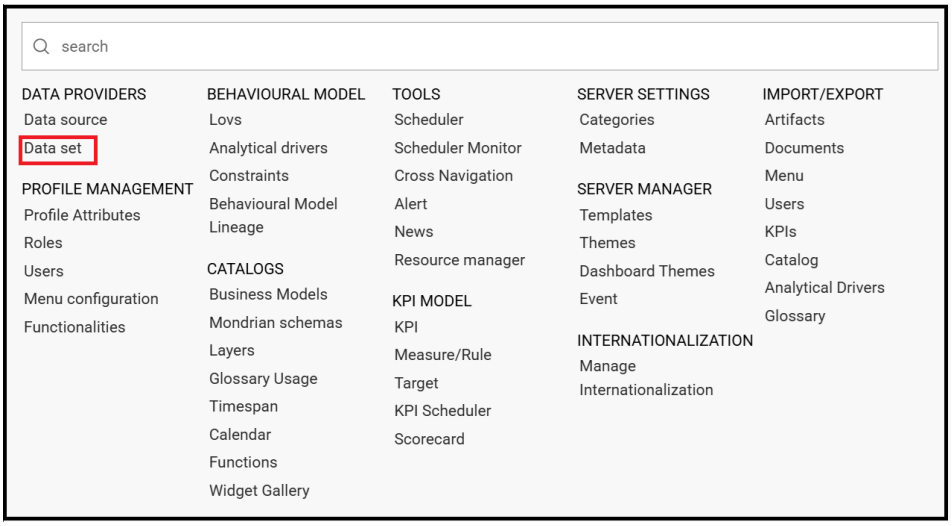

Figura 4.13: Interfaccia grafica di Knowage ufficiale

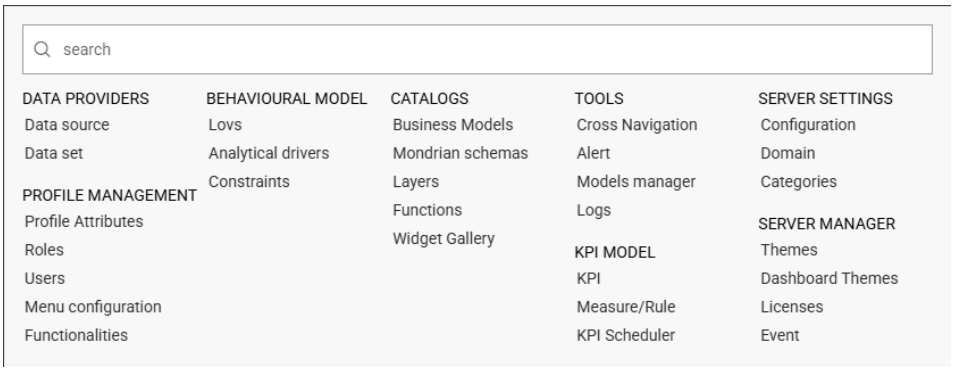

Figura 4.14: Interfaccia grafica di Knowage installata

Da successive verifiche non sono emerse altre versioni disponibili se non quelle già considerate, per cui è stato possibile escludere questa funzionalità.

4.5.3 Knowage Studio

Knowage Studio è un ambiente di sviluppo integrato (*IDE*[g]) per la creazione e la gestione di report e dashboard all'interno della piattaforma Knowage. È progettato per semplificare il processo di sviluppo, consentendo agli utenti di creare report complessi e dashboard interattivi in modo intuitivo.
E' basato su Eclipse e offre un'interfaccia grafica che consente agli sviluppatori di progettare report utilizzando un editor drag-and-drop, senza la necessità di scrivere codice complesso. Knowage Studio supporta l'integrazione con diverse fonti di dati, inclusi database relazionali, file di testo e servizi web, consentendo agli utenti di accedere facilmente ai dati necessari per i loro report. E' stato utilizzato da Knowage fino alla versione 6.2, poi definitivamente

abbandonato a favore di BIRT.
Si è voluto analizzare questo prodotto in quanto poteva dimostrare l apossibilità di deploy di report in knowage tramite un editor grafico. Esso infatti funziona tramite l'aggiunta di un plugin a Eclipse che permette di caricare i report in modo simile a quanto fa JasperStudio per JasperServer.
L'analisi ha evidenziato che questo prodotto non è più supportato da Knowageper cui non è stato possibile testare questa funzionalità.

4.5.4 Report Server

Report Server è una piattaforma di gestione centralizzata dei report che consente agli utenti di creare, gestire e distribuire report in modo semplice ed efficiente. È progettato per essere flessibile e scalabile, consentendo alle organizzazioni di adattare la piattaforma alle proprie esigenze specifiche.
Report Server è stato considerato per la parte server in quanto è un prodotto in grado di eseguire report sia jasper che Birt e quindi favorevole a un'implementazione più veloce e semplice come sostituto di JasperServer. Come nei prodotti precedenti è stata eseguita un'analisi delle funzionalità per verificarne il funzionamento e la possibilità di utilizzo.
Da questa analisi è emerso che Report Server mette a disposizione la possibilità di ospitare ed eseguire report realizzati sia in Jasper che in Birt ma non permette una loro esecuzione dinamica tramite script Javascript ma solo con uno script in linguaggio Groovy, che non è un linguaggio di scripting comune e potrebbe rappresentare una barriera per l'adozione da parte degli utilizzatori.
Essendo questo un limite importante e considerando il target di utenti a cui è rivolto il prodotto, si è deciso di escludere questa soluzione in quanto non soddisfa i requisiti obbligatori definiti in precedenza.

4.5.5 Stimulsoft Designer e Stimulsoft Server

Stimulsoft è una suite di strumenti di reportistica e business intelligence che consente agli utenti di creare report interattivi e dashboard personalizzati. È progettata per essere facile da usare e altamente flessibile, supportando diverse fonti di dati e offrendo un'ampia gamma di funzionalità per la generazione e la visualizzazione dei dati.
Stimulsoft è stato considerato in combinazione tra Stimulsoft Designer e Stimulsoft Server per poter replicare le funzionalità di JasperServer, in quanto offre un editor grafico avanzato per la creazione dei report e un server per la gestione e l'esecuzione degli stessi.
L'ambiente di lavoro si presenta in forma di un'applicazione desktop con grafica che ricorda molto quella di office, che quindi facilità l'utilizzo da parte di utenti non esperti.
Questo, pur essendo un grande punto di forza, rappresenta anche un limite in quanto il prodotto è disponibile solo per ambiente Windows e non è possibile eseguirlo su server Linux, che è l'ambiente di lavoro più comune per questo tipo di applicazioni.
Una modalità alternativa di esecuzione considerata è quella tramite Docker, ma in questo caso non si è riusciti pienamente ad eseguire il prodotto in quanto l'immagine *Docker* ufficiale disponibile non si eseguiva correttamente e quindi limitando la possibilità di testare le funzionalità offerte.

4.6 Conclusioni

A termine di questa analisi decisamente più approfondita dei vari prodotti e delle loro funzionalità, è stato possibile escludere quasi tutte le soluzioni considerate in quanto molte di esse non soddisfaceva completamente i requisiti obbligatori definiti in precedenza o non è

sato possibile verificarne a fondo il funzionamento e quindi le fuzionalità offerte.
Il prodotto tenuto a maggior considerazione per una successiva base di partenza per lo sviluppo di un *PoC* è stato Birt Design Report in quanto open-source in quanto permette una maggior personalizzazione, flessibiltà e facilità di integrazione con altre applicazioni, anche sviluppate internamente, che possono riuscire a replicare le funzionalità ricercate e definite.
Inoltre, Birt Design Report è un prodotto maturo e ampiamente utilizzato, con una vasta comunità di utenti e sviluppatori che contribuiscono al suo sviluppo e supporto. Questo garantisce una maggiore affidabilità e una maggiore disponibilità di risorse per la risoluzione di eventuali problemi o per l'implementazione di nuove funzionalità.

Capitolo 5

Progettazione e realizzazione del Proof of Concept

Questo capitolo descrive lo sviluppo del Proof of Concept (PoC) di reportistica che è stato realizzato durante lo stage analizzandone architettura, tecnologie e funzionalità.

5.1 Obiettivi del PoC

A seguito delle analisi e dei confronti effettuati si è scelto di procedere alla realizzazione di un *PoC* in linguaggio `Java` per un applicativo locale e in linguaggio `Python` per una versione server utilizzando la soluzione open-source Birt Design Report e le sue librerie Birtruntime che permettono di sviluppare un applicativo in grado di compilare report Birt anche in assenza del programma di design.
L'obiettivo del *PoC* è quello di dimostrare la fattibilità tecnica di un sistema di reportistica basato su Birt Design Report e Birtruntime, evidenziando le potenzialità e i vantaggi di questa soluzione rispetto ad altre opzioni considerate.
Il *PoC* è stato progettato per essere un'applicazione semplice ma funzionale, in grado di generare report basati su template predefiniti e di visualizzarli in un formato leggibile.

5.2 Tecnologie e strumenti utilizzati

In questa sezione vengono analizzate le principali tecnologie utilizzate nello sviluppo del progetto. Per ciascuna di esse vengono descritti il ruolo, i punti di forza, le possibili alternative e le motivazioni che hanno portato alla scelta finale.

5.2.1 Birt Design Report

Una piattaforma open-source per la creazione di report, che offre un ambiente di sviluppo integrato e una vasta gamma di funzionalità per la progettazione e la generazione di report. E' la base su cui si è deciso di costruire il sistema di reportistica, grazie alla sua flessibilità e alla possibilità di integrare dati da diverse fonti, inclusi servizi *REST* che restituiscono dati in formato *JSON*, e per il fatto di essere la base di studio del progetto.

5.2.2 Birtruntime

E' una libreria Java che consente di eseguire i report progettati con Birt Design Report in ambienti privi dell'*IDE*, permettendo così l'integrazione con applicazioni esterne.
La scelta di Birtruntime è stata motivata dalla necessità di disporre di uno strumento che consentisse di eseguire i report in modo autonomo, senza dover dipendere dall'ambiente di sviluppo.
Constituisce, assieme a birt design report, la base su cui è stato costruito il sistema di reportistica, permettendo di eseguire i report progettati con Birt Design Report in ambienti privi dell'*IDE*, consentendo così l'integrazione con applicazioni esterne.

5.2.3 Java

Java è un linguaggio di programmazione orientato agli oggetti, progettato per garantire portabilità e affidabilità attraverso l'utilizzo della Java Virtual Machine (*JVM*[g]). Il codice sorgente viene compilato in bytecode, rendendo possibile l'esecuzione su diverse piattaforme senza modifiche.
Uno degli aspetti più rilevanti di *Java* è la sua solidità, dovuta anche alla gestione automatica della memoria tramite garbage collection e alla presenza di un sistema di tipi fortemente strutturato. Inoltre, l'ampio ecosistema di librerie e framework ne favorisce l'impiego in contesti applicativi complessi.
Tra le possibili alternative si possono considerare C++, che offre un maggiore controllo a basso livello ma richiede una gestione manuale delle risorse, e C#, che presenta caratteristiche simili a *Java* ma è maggiormente integrato nell'ambiente Microsoft.
Nel contesto del progetto, *Java* è stato scelto per la sua affidabilità e per la familiarità acquisita durante il percorso accademico. La sua struttura orientata agli oggetti ha inoltre facilitato la realizzazione di componenti modulari e facilmente manutenibili.
Esso è stato utilizzato per implementare il motore di generazione dei report, sfruttando le *API* di BIRT Runtime per eseguire il rendering dei documenti a partire dai file di design.

Figura 5.1: Logo di Java

5.2.4 Python

Python è un linguaggio di programmazione ad alto livello caratterizzato da una sintassi semplice e intuitiva, che lo rende particolarmente adatto allo sviluppo rapido di applicazioni. Negli ultimi anni ha trovato ampia diffusione in diversi ambiti, tra cui sviluppo web, automazione e analisi dei dati.
Uno dei principali vantaggi di *Python* è la leggibilità del codice, che consente di comprendere facilmente anche programmi complessi. A questo si aggiunge la disponibilità di numerose librerie che permettono di estendere rapidamente le funzionalità dell'applicazione senza dover implementare tutto da zero.
Tra le alternative si possono citare JavaScript, utilizzato spesso lato server tramite Node.js, oppure *Java*, che offre prestazioni maggiori ma risulta generalmente più verboso. Un'altra opzione è Go, che garantisce buone performance ma è meno diffuso nel contesto considerato.
La scelta di *Python* è stata dettata principalmente dalla necessità di sviluppare in modo

rapido e mantenere il codice facilmente comprensibile. Questo ha permesso di concentrarsi maggiormente sulla logica applicativa piuttosto che sugli aspetti sintattici del linguaggio.

Figura 5.2: Logo di Python

5.2.5 Flask

Flask è un micro-framework web sviluppato in *Python*, progettato per la realizzazione di applicazioni web leggere e modulari. A differenza di framework più strutturati, fornisce un insieme ridotto di funzionalità di base, lasciando allo sviluppatore la libertà di integrare librerie aggiuntive in base alle esigenze del progetto.
Uno dei principali punti di forza di Flask risiede nella sua semplicità e flessibilità. La struttura minimale consente di sviluppare rapidamente prototipi funzionanti e di mantenere il codice facilmente leggibile e manutenibile. Inoltre, la vasta documentazione disponibile e il supporto della community facilitano la risoluzione di eventuali problematiche.
Tra le alternative più rilevanti vi è FastAPI, anch'esso basato su *Python*, che offre prestazioni superiori grazie al supporto nativo per la programmazione asincrona e include funzionalità avanzate come la generazione automatica della documentazione delle *API*. Un'altra alternativa è Django, un framework completo che integra numerosi strumenti pronti all'uso, ma che risulta più complesso e meno flessibile.
La scelta di Flask è stata motivata dalla necessità di disporre di uno strumento leggero e facilmente configurabile, adeguato alla complessità del progetto. In particolare, non essendo richieste funzionalità avanzate come l'elaborazione asincrona o un'infrastruttura fortemente strutturata, Flask ha rappresentato la soluzione più equilibrata in termini di semplicità, controllo e rapidità di sviluppo.
Esso è stato utilizzato per implementare il server *REST* che gestisce le richieste di generazione dei report, consentendo di esporre un'interfaccia *HTML* semplice e intuitiva per l'interazione con i client.

Figura 5.3: Logo di Flask

5.2.6 Visual Studio Code

Visual Studio Code[1] è un editor di codice sviluppato da Microsoft, ampiamente utilizzato per lo sviluppo software grazie alla sua leggerezza e alla grande possibilità di personalizzazione tramite estensioni.
Uno dei suoi principali punti di forza è la capacità di adattarsi a diversi linguaggi e tecnologie, rendendolo uno strumento versatile all'interno di progetti eterogenei. L'integrazione con Git, il supporto al debugging e le funzionalità di autocompletamento contribuiscono a migliorare significativamente la produttività durante lo sviluppo.
Nel progetto è stato scelto Visual Studio Code in quanto rappresenta un buon compromesso tra semplicità e funzionalità. La possibilità di configurarlo in base alle esigenze specifiche ha reso più efficiente lo sviluppo, soprattutto considerando l'utilizzo di tecnologie differenti.

[1] *Visual Studio Code*. URL: https://code.visualstudio.com/.

Figura 5.4: Logo di Visual Studio Code

5.3 Sviluppo del programma Java

5.3.1 Architettura del programma

L'architettura complessiva del sistema può essere suddivisa nei seguenti livelli logici:

- **View**: rappresentata dalla classe `Window`, gestisce la presentazione dei dati e l'interazione con l'utente;
- **Controller**: rappresentato dalla classe `Controller`, coordina le azioni dell'utente e gestisce il flusso applicativo;
- **Model**: composto dalle classi `ReportConfig` e `BirtDesignToDocument`, contiene la logica di business e i dati necessari alla generazione dei report.

Dal punto di vista architetturale, è stata posta particolare attenzione alla gestione delle risorse, in quanto il motore BIRT richiede una corretta inizializzazione e chiusura per evitare problemi di memoria.
Per questo motivo, ogni ciclo di generazione del report prevede:

- inizializzazione controllata del motore;
- esecuzione isolata del task;
- rilascio esplicito delle risorse.

Questo approccio garantisce stabilità anche in scenari con esecuzioni ripetute.
Come si può notare, è stato adottato un tipo di architettura MVC che consente di separare chiaramente le responsabilità tra la logica di business, la presentazione e il controllo dell'applicazione.
Graficamente l'architettura del programma *Java* e il suo funzionamento sono rappresentati nella figura 5.5.

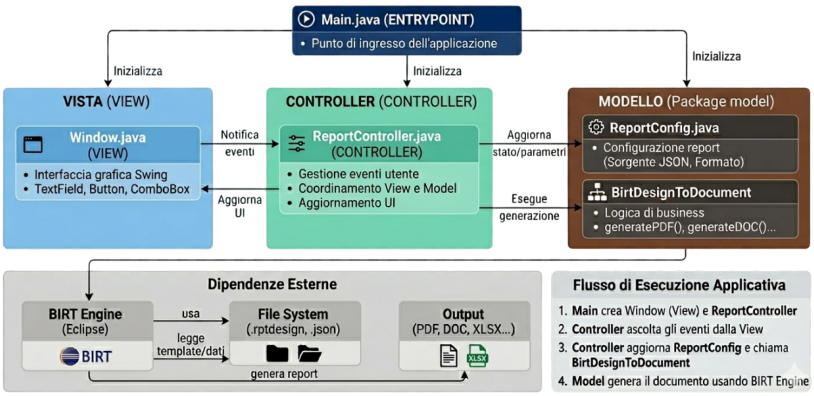

Figura 5.5: Architettura del programma Java

5.3.2 Classe BirtDesignToDocument

La classe `BirtDesignToDocument` rappresenta il componente centrale del programma *Java* sviluppato, in quanto si occupa dell'intero processo di generazione dei documenti a partire da un file di report BIRT. Essa costituisce il punto di integrazione tra l'applicazione *Java* e il motore di rendering fornito dalle librerie BIRT Runtime.
Il suo ruolo principale è quello di orchestrare tutte le fasi necessarie alla produzione del report finale, partendo dal caricamento del file di design fino alla generazione del documento nel formato richiesto. Questa classe incapsula quindi la logica di business legata alla reportistica, separandola dagli aspetti di interfaccia e controllo, in linea con il pattern architetturale Model-View-Controller adottato nel progetto.

Struttura e responsabilità della classe

La classe è progettata per essere riutilizzabile e indipendente dall'interfaccia utente. Le sue responsabilità principali possono essere riassunte nei seguenti punti:

- inizializzazione e configurazione del motore BIRT;
- caricamento del file di design del report (`.rptdesign`);
- gestione dinamica dei parametri di input, in particolare dati *JSON*;
- esecuzione del report;
- rendering del documento in diversi formati di output;
- gestione delle risorse e rilascio della memoria.

5.3.3 Attributi della classe

Gli attributi definiti all'interno della classe rappresentano le informazioni minime necessarie per eseguire correttamente il processo di generazione:

- **sourceJson**: rappresenta la sorgente dei dati *JSON* utilizzati per popolare il report. Può essere un URL di un servizio *REST* oppure il percorso di un file locale;
- **sourceBirt**: contiene il percorso del file di design BIRT (`.rptdesign`), che definisce la struttura e il layout del report;
- **json**: stringa contenente il contenuto dei dati *JSON*, utilizzata nei casi in cui la sorgente sia un file locale e sia necessario caricarne il contenuto in memoria.

Questi attributi consentono di rendere la classe flessibile rispetto alle diverse modalità di acquisizione dei dati.

5.3.4 Metodo principale `generateDocument(String format)`

Il metodo `generateDocument` rappresenta il punto di ingresso principale della classe. Esso accetta come parametro una stringa che identifica il formato di output desiderato (ad esempio *PDF*, DOC, XLSX o *HTML*).
Il comportamento del metodo segue una logica di selezione: in base al formato richiesto, viene invocato uno specifico metodo di generazione. Questo approccio consente di mantenere separata la logica di rendering per ciascun formato, migliorando la leggibilità e la manutenibilità del codice.
Il metodo restituisce un valore booleano:

- `true` se la generazione del report è avvenuta con successo;
- `false` in caso di errore durante una delle fasi del processo.

5.3.5 Fasi del processo di generazione del report

Ogni metodo di generazione segue una sequenza di operazioni ben definita, articolata nelle seguenti fasi:

- Configurazione del motore BIRT La prima fase consiste nella configurazione del motore di esecuzione BIRT attraverso un oggetto `EngineConfig`. In questa fase vengono definiti il percorso della directory di lavoro e la configurazione del logging.
  Successivamente viene avviata la piattaforma BIRT tramite il metodo `Platform.startup()`, che inizializza tutte le componenti necessarie all'esecuzione dei report.

- Creazione del report engine Una volta configurato l'ambiente, viene creato il motore di esecuzione tramite una factory (`IReportEngineFactory`). Il motore (`IReportEngine`) rappresenta il componente principale che consente di aprire i file di design e creare task di esecuzione.

- Caricamento del report design Il file di design viene caricato tramite il metodo `openReportDesign()`, che restituisce un oggetto `IReportRunnable`. Questo oggetto rappresenta il report in forma eseguibile. Successivamente viene creato un task di tipo `IRunAndRenderTask`, che consente di eseguire e renderizzare il report in un'unica operazione.

- Gestione dei parametri *JSON* Una fase fondamentale riguarda la gestione dei parametri di input, in particolare dei dati *JSON*. La logica implementata distingue due casi:
  - se la sorgente è un URL, il parametro *JSON* viene passato direttamente al report;
  - se la sorgente è un file locale, il contenuto viene letto e convertito in stringa, quindi assegnato ai parametri del report.

  Questo approccio consente di supportare sia dati statici che dinamici.

- Configurazione delle opzioni di rendering In base al formato richiesto vengono impostate specifiche opzioni di rendering:
  - `PDFRenderOption` per *PDF*;
  - `RenderOption` per documenti generici;
  - `EXCELRenderOption` per file Excel;
  - `HTMLRenderOption` per HTML.

  In questa fase viene inoltre definito il nome del file di output, spesso arricchito con un timestamp per evitare sovrascritture.

- Esecuzione del report Il task viene eseguito tramite il metodo `run()`, che avvia il processo di generazione del documento. Durante questa fase il motore BIRT elabora i dati e produce il file finale.

- Rilascio delle risorse Al termine dell'esecuzione, vengono rilasciate le risorse utilizzate:
  - chiusura del task;
  - distruzione del report engine;
  - arresto della piattaforma BIRT.

  Questa fase è fondamentale per evitare problemi di memoria e garantire la stabilità dell'applicazione.

Lo schema di flusso del processo di generazione è rappresentato nella figura 5.6.

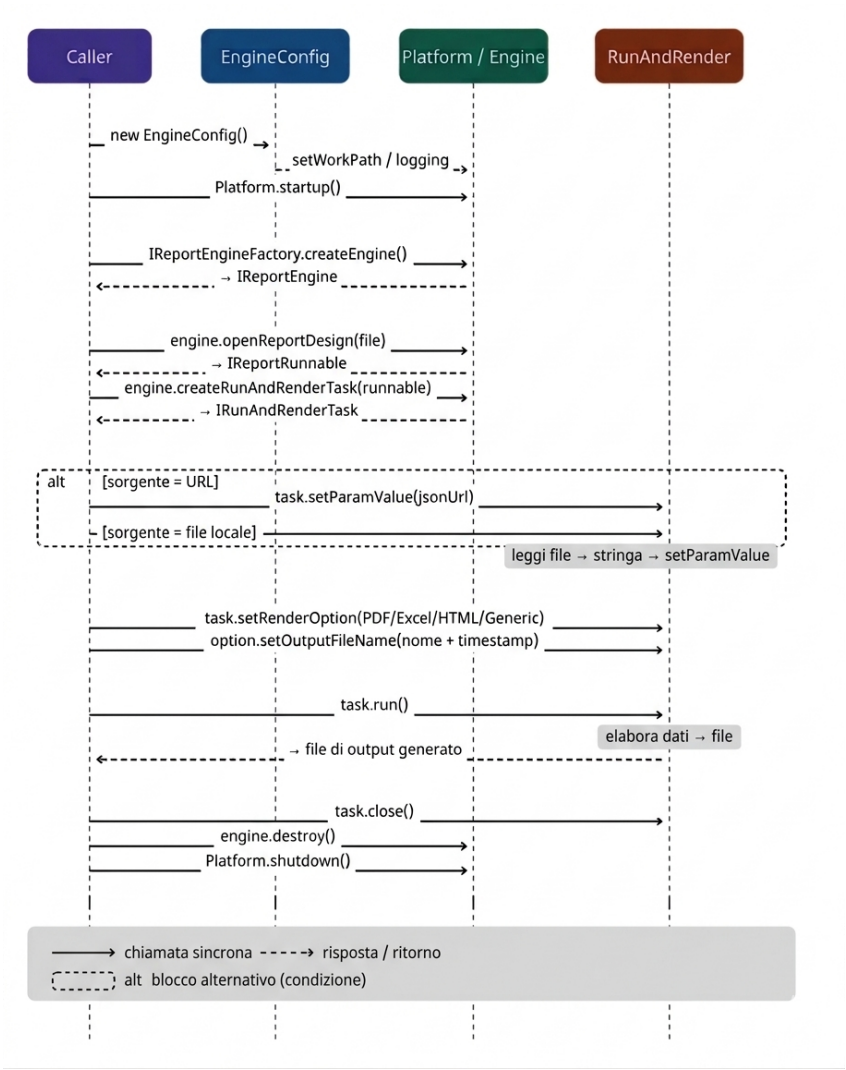

Figura 5.6: Flusso di generazione del report

5.3.6 Considerazioni progettuali

La progettazione della classe segue principi fondamentali dell'ingegneria del software:

- separazione delle responsabilità;
- modularità;
- estendibilità;
- manutenibilità.

L'utilizzo delle *API* ufficiali di BIRT consente inoltre di garantire affidabilità e compatibilità con il framework.
Graficamente l'applicativo si presenta come in figura 5.7, con un'interfaccia semplice che consente di selezionare il file di design, specificare la sorgente dei dati e avviare il processo di generazione del report.

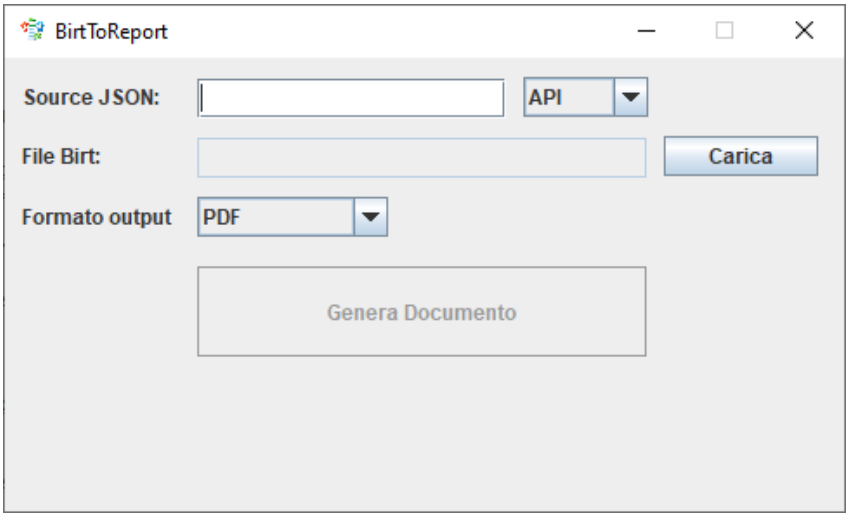

Figura 5.7: Applicativo Java per la generazione dei report

5.4 Architettura del PoC

La soluzione sviluppata per la realizzazione di un report server basato su BIRT adotta un'architettura di tipo *client–server*, progettata per consentire la generazione centralizzata di report a partire da file di design e da sorgenti dati esterne. L'obiettivo principale dell'architettura è quello di separare la logica di generazione dei report dalla gestione delle richieste provenienti dai client, rendendo il sistema modulare, facilmente estendibile e integrabile con applicazioni esterne. Il sistema consente di generare report in modo dinamico a partire da un file di definizione del report nel formato `.rptdesign` e da una sorgente dati esposta tramite API REST in formato *JSON*. La richiesta di generazione del report viene inviata da un client ad un server REST, il quale si occupa di orchestrare il processo di generazione del documento e di restituire il file finale al richiedente. L'architettura adottata è composta da due componenti principali: un motore di generazione dei report basato su *BIRT Runtime*, sviluppato in linguaggio *Java*, e un server REST implementato in *Python* mediante il framework Flask. Questa suddivisione consente di sfruttare le funzionalità offerte dalla piattaforma BIRT per la generazione dei report mantenendo al contempo un'interfaccia di accesso semplice e standardizzata tramite protocollo *HTML*.

5.4.1 Struttura del PoC

Il *PoC* è organizzato in due moduli principali, uno dedicato al motore di generazione dei report e l'altro al server REST. La struttura del *PoC* è la seguente:

```
BirtReportServer/
|-- src/com/report/model/BirtReportEngine.java
|-- bin/com/report/model/BirtReportEngine.class
|-- bin/com/report/model/BirtDesignToDocument.class
|-- lib/*.jar
|-- server.py
|-- requirements.txt
|-- test-client.html
|-- setup.bat / setup.sh
|-- compile.bat / compile.sh
|-- start-server.bat / start-server.sh
\-- install.bat / install.sh
```

```
Runtime (auto-create)
C:\Users\<user>\reports\
|-- uploads/
|-- output/
|-- birt/
\-- logs/
    \-- server.log
```

La struttura del *PoC* è organizzata in modo modulare per separare chiaramente le diverse componenti del sistema. La directory principale `BirtReportServer` contiene sia il codice sorgente necessario alla generazione dei report sia gli script utilizzati per la compilazione, l'installazione e l'avvio del server.
La cartella `src` contiene il codice sorgente *Java* che implementa il wrapper per il motore di reporting BIRT. In particolare, la classe `BirtReportEngine.java` incapsula le funzionalità principali del motore BIRT e fornisce un'interfaccia semplificata per la generazione dei report.
La directory `bin` ospita le classi *Java* compilate, generate durante la fase di build del progetto. Queste classi vengono utilizzate dal server *Python* per interagire con il motore di reporting BIRT tramite il runtime *Java*.
La cartella `lib` contiene le librerie esterne necessarie al funzionamento del sistema, in particolare i file `.jar` appartenenti alla distribuzione di BIRT versione 4.21.
Il file `server.py` implementa il server REST sviluppato in *Python* utilizzando il framework Flask. Questo componente espone le *API* che permettono ai client di inviare richieste di generazione dei report e di ottenere i documenti prodotti dal motore BIRT.
Il file `requirements.txt` elenca le dipendenze *Python* necessarie per eseguire il server, facilitando l'installazione dell'ambiente di esecuzione.
Il file `test-client.html` è un semplice client web che consente di testare le funzionalità del server REST inviando richieste di generazione dei report e visualizzando i risultati ottenuti.
Gli altri file presenti nella directory principale comprendono script di utilità per il setup dell'ambiente (`setup`), la compilazione delle classi *Java* (`compile`), l'avvio del server (`start-server`) e l'installazione delle dipendenze *Python* (`install`). Tali script sono forniti sia per ambienti Windows che Unix-like al fine di garantire la portabilità del sistema.
Durante l'esecuzione del server viene inoltre creata automaticamente una directory di lavoro all'interno del percorso `reports` nella home dell'utente. Questa directory contiene diverse sottocartelle utilizzate per la gestione dei file temporanei (`uploads`), dei report generati (`output`), della configurazione del motore BIRT (`birt`) e dei file di log del server (`logs`), utili per il monitoraggio e il debug del sistema.

5.5 Componenti principali

L'architettura del sistema è composta da diversi componenti che collaborano tra loro per gestire il processo di generazione dei report.

5.5.1 Motore di generazione dei report (Java / BIRT Runtime)

Il motore di generazione dei report è implementato in linguaggio *Java* e utilizza la libreria *BIRT Runtime* per l'esecuzione dei report progettati tramite *BIRT Designer*. Questo componente rappresenta il nucleo del sistema ed è responsabile della compilazione e della generazione dei documenti a partire dai file di design. In particolare, il motore di generazione si occupa delle seguenti attività:

- caricamento del file di definizione del report (`.rptdesign`);

- recupero dei dati dalla sorgente esterna, nel caso specifico un'API REST che restituisce dati in formato *JSON*;
- elaborazione del report secondo il layout definito nel file di design;
- generazione del documento finale nel formato richiesto, ad esempio *PDF* o Excel.

Il codice *Java* sviluppato per il *Proof of Concept* locale è stato riutilizzato e adattato per consentire l'esecuzione del processo di generazione dei report tramite linea di comando, rendendo possibile l'integrazione con altri componenti del sistema.

5.5.2 Server REST (Python / Flask)

Il server REST rappresenta il punto di accesso principale al sistema di reportistica ed è stato sviluppato utilizzando il framework Flask per *Python*. Questo componente svolge il ruolo di orchestratore delle richieste e si occupa di gestire l'interazione tra il client e il motore di generazione dei report. Il server espone un'interfaccia REST che consente ai client di richiedere la generazione di un report specificando i parametri necessari. Tra le principali responsabilità del server vi sono:

- ricezione delle richieste *HTML* inviate dai client;
- validazione dei parametri forniti nella richiesta;
- gestione dei file temporanei necessari alla generazione del report;
- invocazione del processo *Java* responsabile della compilazione del report;
- restituzione del file generato come risposta alla richiesta del client.

Per eseguire il motore di generazione dei report, il server Python avvia un processo *Java* esterno utilizzando il modulo `subprocess`. In questo modo è possibile integrare componenti sviluppati in linguaggi diversi mantenendo una chiara separazione tra le responsabilità del sistema.

5.5.3 Client

Il client rappresenta qualsiasi applicazione in grado di inviare richieste *HTML* al server REST. Può trattarsi di un browser web, di un'applicazione desktop oppure di un altro servizio applicativo. Il client invia una richiesta di generazione del report specificando:

- il file di definizione del report;
- l'indirizzo dell'*API* da cui recuperare i dati;
- il formato di output desiderato.

Una volta completato il processo di generazione, il client riceve il documento prodotto dal server come risposta *HTML*.

5.5.4 Vantaggi dell'architettura adottata

L'architettura proposta presenta diversi vantaggi dal punto di vista della modularità e dell'integrazione con sistemi esterni. In primo luogo, la separazione tra il motore di generazione dei report e il server REST consente di isolare la logica di business relativa alla creazione dei report dalla gestione delle richieste *HTML*. Questa scelta progettuale migliora la manutenibilità del sistema e facilita eventuali modifiche o estensioni future. Un ulteriore vantaggio è rappresentato dall'utilizzo di un'interfaccia REST standardizzata, che consente

di integrare facilmente il servizio di reportistica con altre applicazioni o servizi software. Qualsiasi sistema in grado di effettuare chiamate *HTML* può infatti richiedere la generazione di un report. Infine, il riutilizzo del codice *Java* sviluppato durante il *Proof of Concept* locale ha consentito di ridurre i tempi di sviluppo e di mantenere la compatibilità con i report già progettati utilizzando *BIRT Designer*, rendendo la soluzione proposta particolarmente adatta a scenari di integrazione con sistemi esistenti. Graficamente l'architettura complessiva del sistema è rappresentata nella figura 5.8, che illustra le componenti principali e le interazioni tra di esse.

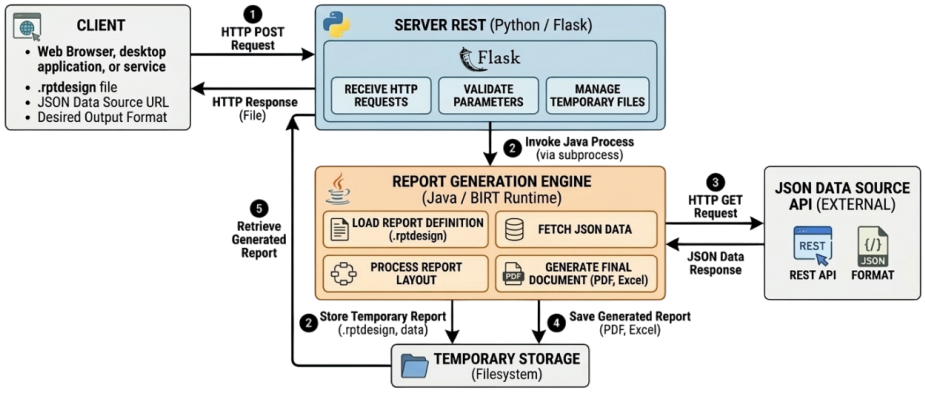

Figura 5.8: Architettura della soluzione proposta

5.6 Implementazione della soluzione

Una volta definita l'architettura generale del sistema, è stata realizzata una prima implementazione con l'obiettivo di verificare la fattibilità tecnica della soluzione proposta.
L'implementazione sviluppata costituisce un *Proof of Concept* (*PoC*) di un sistema di reportistica centralizzato basato sul motore di reporting BIRT.
Lo scopo principale di questa fase è stato quello di costruire un prototipo funzionante in grado di generare report dinamicamente a partire da file di design `.rptdesign` e da sorgenti dati esterne esposte tramite API REST.
Il sistema è stato progettato per separare chiaramente la logica di generazione dei report dal livello di esposizione dei servizi, così da rendere la soluzione modulare e facilmente integrabile in contesti applicativi differenti.
Dal punto di vista architetturale, l'implementazione si basa su due componenti principali: un motore di generazione dei report sviluppato in linguaggio *Java* e un server REST implementato in *Python* mediante il framework Flask.
Il motore *Java* è responsabile dell'esecuzione del processo di rendering dei report tramite le librerie BIRT, mentre il server *Python* funge da interfaccia tra i client e il motore di generazione, gestendo le richieste e coordinando l'esecuzione delle operazioni necessarie.

5.6.1 Motore di generazione dei report

Il cuore del sistema è rappresentato dal motore di generazione dei report sviluppato in linguaggio *Java*.
Questo componente utilizza le librerie del runtime di BIRT per eseguire il rendering dei report definiti tramite file di design `.rptdesign`.

Il motore è stato progettato in modo da poter essere eseguito tramite linea di comando, consentendo così l'integrazione con altri servizi o applicazioni.
La logica principale è stata incapsulata all'interno della classe `BirtReportWrapper`, che rappresenta il punto di ingresso dell'applicazione.
Questa classe ha il compito di ricevere e validare i parametri necessari alla generazione del report, tra cui il percorso del file di design, l'URL della sorgente dati e il formato di output desiderato.

```
package com.report.model;

public class BirtReportWrapper {

    public static void main(String[] args) {
        // Entry point per esecuzione da linea di comando
    }

}
```

Il metodo `main` rappresenta l'entry point dell'applicazione e viene invocato quando il motore viene eseguito tramite riga di comando.
In questa fase vengono verificati i parametri forniti dall'utente e viene avviato il processo di generazione del report.
La logica operativa del sistema è invece implementata nella classe `BirtDesignToDocument`.
Questa classe si occupa della gestione del processo di generazione del documento e utilizza le *API* del motore BIRT per eseguire il rendering del report.

```
class BirtDesignToDocument {

    private final String sourceJson;
    private final String sourceBirt;
    private final String outputDir;
    private final String birtHome;

    public String generateDocument(String format) throws Exception {
        // Logica di generazione del report
    }
}
```

Durante l'esecuzione del metodo di generazione, il sistema effettua una serie di operazioni successive.
In primo luogo viene caricato il file di design del report, che definisce la struttura del documento e le modalità di visualizzazione dei dati.
Successivamente il motore recupera i dati dalla sorgente *JSON* indicata nei parametri di esecuzione, consentendo così la generazione di report dinamici basati su dati aggiornati.
Una volta ottenuti i dati necessari, il motore BIRT procede con la fase di rendering del report e genera il documento finale nel formato richiesto.
Il sistema supporta diversi formati di esportazione, tra cui *PDF* e *HTML*, che rappresentano i formati più comuni per la distribuzione dei report.
La separazione tra la classe di avvio e la classe responsabile della generazione dei documenti consente di mantenere una struttura del codice più modulare e facilita eventuali estensioni future del sistema.

5.6.2 Implementazione del server REST

Per rendere il motore di generazione accessibile da applicazioni esterne è stato sviluppato un server REST in linguaggio *Python* utilizzando il framework Flask.

Questo componente svolge il ruolo di interfaccia tra i client e il motore di generazione dei report implementato in *Java*.
Il server REST riceve le richieste dei client tramite API *HTML* e si occupa di gestire l'intero flusso di generazione del report.
In particolare, il server valida i parametri ricevuti, prepara l'ambiente di esecuzione e avvia il processo *Java* responsabile della generazione del documento.
La configurazione iniziale del server definisce le directory utilizzate per la gestione dei file caricati, dei report generati e dei log di sistema.

```
BASE_DIR = Path.home() / "reports"
UPLOAD_DIR = BASE_DIR / "uploads"
OUTPUT_DIR = BASE_DIR / "output"
BIRT_HOME = BASE_DIR / "birt"
LOG_DIR = BASE_DIR / "logs"

app = Flask(__name__)
CORS(app)
```

Questa organizzazione delle directory consente di mantenere separati i diversi elementi del sistema, facilitando la gestione dei file temporanei e dei documenti generati.
In particolare, la directory di upload viene utilizzata per memorizzare temporaneamente i file di design dei report caricati dai client, mentre la directory di output contiene i documenti generati dal motore BIRT.

5.6.3 Definizione delle API REST

Il server implementa una serie di endpoint *REST* che consentono ai client di interagire con il sistema di reportistica. Queste *API* permettono di verificare lo stato del servizio e di richiedere la generazione di nuovi report.
Nella tabella 5.1 sono elencati gli endpoint esposti dal server REST:

Endpoint	Descrizione
`/api/reports/health`	Controllo dello stato del server
`/api/reports/generate`	Generazione di un nuovo report

Tabella 5.1: Endpoint REST del server di reportistica

Il primo endpoint è dedicato al controllo dello stato del server e consente di verificare rapidamente se il servizio è attivo.

```
@app.route('/api/reports/health', methods=['GET'])
def health_check():
    return {"status": "ok"}
```

Il secondo endpoint del sistema è invece quello dedicato alla generazione dei report.
Questo endpoint riceve una richiesta *HTML* contenente i parametri necessari alla generazione del documento, tra cui il file di design del report, l'URL della sorgente dati e il formato di output desiderato.

```
@app.route('/api/reports/generate', methods=['POST'])
def generate_report():
    # gestione richiesta e avvio processo Java
```

Quando una richiesta di generazione viene ricevuta, il server esegue una serie di operazioni sequenziali.
In primo luogo il file di design del report viene salvato nella directory di upload.

Successivamente vengono estratti i parametri necessari all'esecuzione del motore BIRT, tra cui l'indirizzo della sorgente dati e il formato di output.
Una volta completata la fase di preparazione, il server avvia il processo *Java* tramite il modulo `subprocess` di *Python*.
Questo approccio consente di mantenere separati i due ambienti di esecuzione e di riutilizzare il motore *Java* come componente indipendente.
L'esecuzione del motore avviene tramite una chiamata a riga di comando simile alla seguente:

```
java -cp classpath \
com.report.model.BirtReportWrapper \
report.rptdesign \
https://api.example.com/data \
output_dir \
birt_home \
PDF
```

Al termine dell'esecuzione, il documento generato viene restituito al client come risposta alla richiesta *HTML*.

5.6.4 Client di test

Durante la fase di sviluppo è stato realizzato anche un semplice client web di test, con lo scopo di facilitare la verifica del funzionamento del sistema. Questo client, implementato come una pagina *HTML* statica, consente di inviare richieste al server REST tramite un'interfaccia grafica.
Attraverso questa interfaccia l'utente può selezionare il file di design del report, specificare l'URL della sorgente dati e scegliere il formato di output desiderato. Una volta inviati i parametri, il client invia la richiesta al server REST e consente di scaricare il report generato.
L'utilizzo di un client di test si è rivelato particolarmente utile durante lo sviluppo del Proof of Concept, poiché ha permesso di verificare rapidamente il corretto funzionamento delle *API* e del processo di generazione dei report senza dover ricorrere a strumenti esterni.
Graficamente, il client di test si presenta come in figura 5.9, con un'interfaccia semplice e intuitiva che consente di interagire facilmente con il sistema di reportistica.

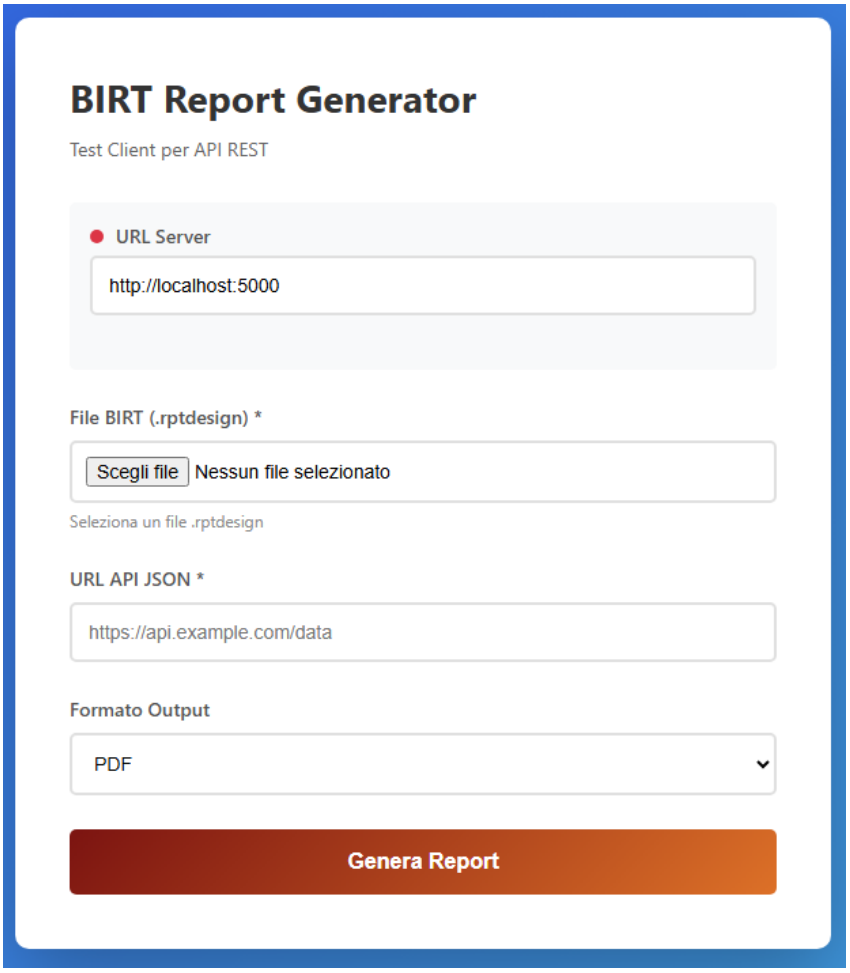

Figura 5.9: Client di test per il server REST

5.6.5 Flusso di esecuzione della richiesta

Per comprendere meglio il funzionamento complessivo del sistema sviluppato, è utile analizzare il flusso di esecuzione che porta alla generazione di un report a partire da una richiesta inviata da un client.
Il processo coinvolge diversi componenti dell'architettura, tra cui il client applicativo, il server REST sviluppato in *Python* e il motore di generazione dei report implementato in *Java* tramite le librerie BIRT.
Il flusso di esecuzione può essere suddiviso in una serie di fasi successive, ciascuna delle quali svolge un ruolo specifico all'interno del processo di generazione del report.

1. **Invio della richiesta da parte del client**

   Il processo ha inizio quando un client invia una richiesta *HTML* al server REST tramite l'endpoint dedicato alla generazione dei report.
   La richiesta contiene i parametri necessari alla generazione del documento, tra cui il file di design del report in formato `.rptdesign`, l'URL della sorgente dati da cui recuperare le informazioni e il formato di output desiderato.
   Nel contesto del Proof of Concept sviluppato, questa richiesta può essere inviata tramite il client web di test oppure tramite strumenti di interrogazione delle *API* come *Postman* o *curl*.

2. **Ricezione e validazione della richiesta**

   Una volta ricevuta la richiesta, il server REST sviluppato in *Python* tramite il framework Flask procede alla validazione dei parametri forniti dal client.
   In questa fase vengono controllati la presenza dei parametri obbligatori e la correttezza del file di design del report.
   Il file `.rptdesign` viene quindi salvato temporaneamente nella directory di upload del server, mentre gli altri parametri della richiesta vengono preparati per l'esecuzione del processo di generazione.

3. **Preparazione dell'ambiente di esecuzione**

   Successivamente il server prepara l'ambiente necessario all'esecuzione del motore BIRT. Questa fase include la definizione dei percorsi delle directory utilizzate dal sistema, tra cui la directory di output in cui verrà salvato il report generato e la directory contenente il runtime di BIRT.
   Una volta completata questa fase, il server costruisce il comando che verrà utilizzato per avviare il motore di generazione dei report implementato in *Java*.

4. **Avvio del motore di generazione dei report**

   Il server REST avvia quindi il motore di generazione tramite una chiamata a riga di comando utilizzando il modulo `subprocess` di *Python*.
   Il comando invoca la classe principale del motore *Java* (`BirtReportWrapper`) e passa come parametri il file di design del report, l'URL della sorgente dati, il percorso della directory di output e il formato di esportazione richiesto.
   Durante questa fase il motore BIRT carica il file di design del report, recupera i dati dalla sorgente *JSON* indicata e avvia il processo di rendering del documento.

5. **Generazione del report**

   Una volta ottenuti i dati necessari, il motore BIRT procede con la generazione del documento finale.
   Il report viene costruito sulla base della struttura definita nel file `.rptdesign` e viene quindi esportato nel formato richiesto dal client, ad esempio *PDF* o *HTML*.
   Il documento generato viene salvato nella directory di output configurata dal server REST.

6. **Restituzione del risultato al client**

   Al termine del processo di generazione, il server REST recupera il file generato e lo restituisce al client come risposta alla richiesta *HTML*.
   Il client può quindi scaricare il documento oppure visualizzarlo direttamente all'interno dell'applicazione che ha effettuato la richiesta.

Questo flusso di esecuzione evidenzia la separazione tra il livello di esposizione dei servizi REST e il motore di generazione dei report.
Tale separazione consente di mantenere il sistema modulare e facilita l'integrazione della soluzione in contesti applicativi differenti, permettendo a diverse applicazioni di utilizzare il servizio di generazione dei report tramite semplici chiamate *API*.
Graficamente viene rappresentato nella figura 5.10.

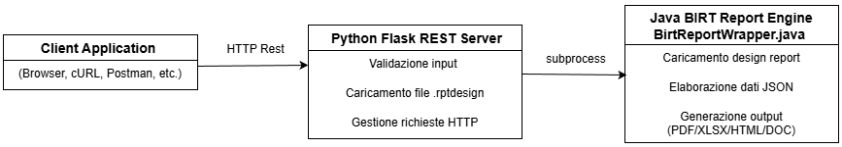

Figura 5.10: Flusso di esecuzione del Proof of Concept

Capitolo 6

Valutazione della soluzione

Il presente capitolo ha l'obiettivo di analizzare e confrontare le prestazioni del sistema sviluppato nel PoC rispetto alla soluzione di riferimento, rappresentata da JasperServer. L'analisi prestazionale riveste un ruolo fondamentale nel processo di valutazione, in quanto consente di verificare non solo la correttezza funzionale delle soluzioni, ma anche la loro efficienza e scalabilità in scenari di utilizzo realistici. A differenza dei test funzionali, che si limitano a verificare la presenza delle caratteristiche richieste, i test prestazionali permettono di valutare il comportamento del sistema in termini di tempi di risposta, gestione delle risorse e capacità di adattarsi a volumi crescenti di dati.

6.1 Obiettivi dei test prestazionali

I test condotti hanno lo scopo di fornire una valutazione preliminare delle prestazioni delle soluzioni analizzate, con particolare riferimento ai seguenti aspetti:

- tempo di generazione dei report;
- comportamento del sistema al variare della dimensione dei dati in ingresso;
- differenze tra esecuzione locale e su server;
- impatto dell'inizializzazione del motore di reporting;
- stabilità delle prestazioni su esecuzioni ripetute.

È importante sottolineare che tali test non hanno l'obiettivo di simulare un ambiente di produzione reale, ma piuttosto di fornire indicazioni comparative utili a supportare la valutazione delle soluzioni.

6.2 Metodologia di test

Per garantire la comparabilità dei risultati, è stato definito un approccio metodologico uniforme per entrambe le soluzioni analizzate.
In primo luogo, si è cercato di mantenere condizioni di esecuzione il più possibile simili, pur tenendo conto delle differenze architetturali tra i sistemi. In particolare:

- il *PoC* è stato eseguito come applicazione locale;
- JasperServer è stato eseguito all'interno di un container Docker;
- entrambi i sistemi hanno utilizzato gli stessi file di report e le stesse sorgenti dati in formato *JSON*.

La scelta di utilizzare dati *JSON* come input è coerente con i requisiti definiti nella fase di analisi, in quanto rappresentano una modalità comune di scambio dati tra servizi applicativi.

6.3 Strumenti utilizzati

Per la raccolta delle metriche prestazionali sono stati utilizzati strumenti differenti, in base alla natura delle soluzioni analizzate:

- per JasperServer è stato utilizzato il pannello *Network* degli strumenti di sviluppo del browser Google Chrome, che consente di misurare i tempi di risposta delle richieste *HTML*;
- per il *PoC* è stato utilizzato VisualVM[1], uno strumento open-source per il monitoraggio delle applicazioni *Java*, utile per osservare tempi di esecuzione e comportamento del sistema.

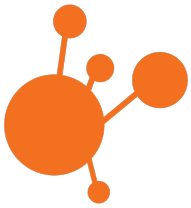

Figura 6.1: Logo di VisualVM

Questa scelta è stata dettata dalla diversa modalità di esecuzione delle due soluzioni, una basata su server e l'altra su applicazione locale.

6.4 Scenari di test

Sono stati definiti diversi scenari di test, variando la complessità del report e la dimensione dei dati in ingresso. In particolare, sono stati considerati:

- report semplice con dataset ridotto (circa 10 elementi);
- report di complessità intermedia con dataset medio (circa 100 elementi);
- report complesso con dataset esteso (circa 1000 elementi);
- esecuzione senza dati (report vuoto).

Questa varietà di scenari consente di osservare il comportamento del sistema al crescere della quantità di dati e della complessità computazionale.

6.5 Risultati dei test

6.5.1 Primo scenario: dataset ridotto

Nel primo scenario è stato utilizzato un report semplice alimentato da un dataset *JSON* contenente circa 10 elementi.
I risultati hanno evidenziato che:

[1] *VisualVM - Visual Tool for Java.* URL: https://visualvm.github.io/.

- la prima esecuzione di JasperServer presenta un tempo significativamente più elevato (circa 18 secondi), dovuto principalmente all'inizializzazione del sistema e al caricamento delle risorse;
- le esecuzioni successive risultano molto più rapide, con tempi nell'ordine del secondo;
- il *PoC* mostra un comportamento simile nelle esecuzioni successive, ma con un tempo iniziale leggermente inferiore.

Questo comportamento è coerente con la presenza di meccanismi di caching e inizializzazione nei sistemi server-based.
Di seguito è riportata una grafica per Jasper server (fig. 6.2) e una grafica del *PoC* (fig. **??**) che confronta i tempi di generazione dei report per entrambe le soluzioni nel primo scenario:

test.pdf?json=https://jsonplace...	200	docume...	Other	0.5 kB	17.98 s
test.pdf?json=https://jsonplace...	200	docume...	Other	0.4 kB	1.19 s
test.pdf?json=https://jsonplace...	200	docume...	Other	0.4 kB	788 ms

Figura 6.2: Tempi di generazione dei report JasperServer nel primo scenario

SwingWorker-pool-1-thread-1	13.191 ms	(100%)	12.886 ms	(100%)
SwingWorker-pool-1-thread-4	705 ms	(100%)	593 ms	(100%)
SwingWorker-pool-1-thread-2	693 ms	(100%)	693 ms	(100%)

Figura 6.3: Tempi di generazione dei report PoC nel primo scenario

6.5.2 Secondo scenario: dataset medio

Nel secondo scenario, caratterizzato da un dataset di circa 100 elementi, si osserva un aumento dei tempi di generazione per entrambe le soluzioni.
Il *PoC* mantiene tempi di risposta comparabili con JasperServer nelle esecuzioni successive, mentre la prima esecuzione continua a rappresentare il caso più oneroso in termini computazionali.
Di seguito è riportata una grafica per Jasper server (fig. 6.4) e una grafica del *PoC* (fig. 6.5) che confronta i tempi di generazione dei report per entrambe le soluzioni nel secondo scenario:

Name	Type	Size	Time
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	16.55 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	1.53 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	994 ms

Figura 6.4: Tempi di generazione dei report JasperServer nel secondo scenario

Name	Total Time	
SwingWorker-pool-1-thread-1	9.497 ms	(100%)
SwingWorker-pool-1-thread-2	1.587 ms	(100%)
SwingWorker-pool-1-thread-3	1.194 ms	(100%)

Figura 6.5: Tempi di generazione dei report PoC nel secondo scenario

6.5.3 Terzo scenario: dataset esteso

Nel terzo scenario, con dataset di circa 1000 elementi, emerge in modo più evidente l'impatto della dimensione dei dati sulle prestazioni.
Entrambe le soluzioni mostrano un incremento significativo dei tempi di elaborazione, evidenziando come la fase di parsing e gestione dei dati *JSON* rappresenti uno dei principali colli di bottiglia.
Nonostante ciò, il comportamento del *PoC* rimane stabile e in linea con quello della soluzione di riferimento.
Di seguito è riportata una grafica per Jasper server (fig. 6.6)e una grafica del *PoC* (fig. 6.7) che confronta i tempi di generazione dei report per entrambe le soluzioni nel terzo scenario:

Name	Type	Size	Time
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	7.98 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	3.67 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	2.19 s

Figura 6.6: Tempi di generazione dei report JasperServer nel terzo scenario

Name	Total Time		Total Time (CPU)	
SwingWorker-pool-1-thread-1	26.903 ms	(100%)	26.104 ms	(100%)
SwingWorker-pool-1-thread-2	3.195 ms	(100%)	2.697 ms	(100%)
SwingWorker-pool-1-thread-3	2.792 ms	(100%)	2.273 ms	(100%)

Figura 6.7: Tempi di generazione dei report PoC nel terzo scenario

6.5.4 Quarto scenario: report senza dati

Nel caso di report senza dati, i tempi di generazione risultano estremamente ridotti, evidenziando come il costo computazionale sia fortemente influenzato dalla quantità di dati da elaborare piuttosto che dalla struttura del report stesso.
Di seguito è riportata una grafica per Jasper server (fig. 6.8) e una grafica del *PoC* (fig. 6.9) che confronta i tempi di generazione dei report per entrambe le soluzioni nel quarto scenario:

Name	Type	Size	Time
test.pdf	document	0.4 kB	9.65 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	2.46 s
test.pdf?json=https://2cc1de8b-6155-4fad-bf6a...	document	0.4 kB	2.17 s

Figura 6.8: Tempi di generazione dei report JasperServer nel quarto scenario

Name	Total Time		Total Time (CPU)	
SwingWorker-pool-1-thread-1	7.600 ms	(100%)	7.600 ms	(100%)
SwingWorker-pool-1-thread-2	5.674 ms	(100%)	4.998 ms	(100%)
SwingWorker-pool-1-thread-3	3.504 ms	(100%)	2.798 ms	(100%)

Figura 6.9: Tempi di generazione dei report PoC nel quarto scenario

6.6 Analisi dei risultati

Dall'analisi complessiva dei risultati emergono alcune considerazioni rilevanti.
In primo luogo, il tempo di inizializzazione rappresenta un fattore critico, in particolare per

JasperServer, che richiede un overhead maggiore a causa della sua architettura basata su container e servizi.
In secondo luogo, le prestazioni delle due soluzioni risultano comparabili nelle esecuzioni successive alla prima, suggerendo che il *PoC* sviluppato è in grado di competere, almeno in termini di tempi di generazione, con una soluzione consolidata come JasperServer.
Un ulteriore elemento da considerare è l'impatto della dimensione dei dati: all'aumentare del dataset, i tempi di elaborazione crescono in modo significativo, evidenziando la necessità di ottimizzazioni, soprattutto nella fase di parsing dei dati *JSON*.
Di seguito vengono riassiunti nella tabella 6.1 i tempi medi di generazione dei report per entrambe le soluzioni nei diversi scenari testati:

Tabella 6.1: Tempi medi di generazione dei report nei diversi scenari

Scenario	JasperServer	PoC
Primo	6,652 s	4,724 s
Secondo	6,358 s	4,092 s
Terzo	4,613 s	10,358 s
Quarto	4,76 s	5,132 s

6.7 Limiti dei test

È opportuno evidenziare alcuni limiti della sperimentazione condotta:

- ambiente di test non equivalente a un contesto di produzione;
- differenze architetturali tra le soluzioni (locale vs container);
- numero limitato di scenari testati;
- assenza di test sotto carico concorrente.

Questi fattori possono influenzare i risultati e devono essere considerati nell'interpretazione dei dati.

6.8 Conclusioni

I risultati ottenuti dimostrano che il Proof of Concept è in grado di garantire prestazioni comparabili a JasperServer in scenari di utilizzo standard, soprattutto nelle esecuzioni successive alla prima.
Nonostante alcune differenze dovute all'architettura e all'ambiente di esecuzione, il sistema sviluppato si dimostra adeguato per l'obiettivo prefissato, ovvero la realizzazione di una soluzione alternativa per la generazione di report.
I risultati ottenuti forniscono quindi una base solida per eventuali sviluppi futuri, nei quali sarà possibile approfondire aspetti legati alla scalabilità, all'ottimizzazione delle prestazioni e all'integrazione in contesti applicativi più complessi.

Capitolo 7

Conclusioni

Il presente elaborato ha avuto come obiettivo principale l'analisi e la valutazione di soluzioni per la generazione di report, con particolare attenzione alla possibilità di realizzare un sistema personalizzato basato sull'utilizzo del motore BIRT. A partire da un'analisi preliminare delle principali piattaforme di reporting, è stato progettato e sviluppato un *PoC* volto a dimostrare la fattibilità di un approccio alternativo rispetto a soluzioni consolidate come JasperServer.
Il lavoro svolto ha previsto diverse fasi: una prima fase di studio delle tecnologie esistenti, una fase di progettazione dell'architettura software e infine una fase di implementazione e testing. In particolare, è stata sviluppata la componente centrale `BirtDesignToDocument`, responsabile della generazione dei documenti a partire da file di design BIRT e dati in formato *JSON*, integrata all'interno di un'architettura di tipo Model-View-Controller.

7.1 Risultati ottenuti

I risultati ottenuti dimostrano che il Proof of Concept sviluppato è in grado di soddisfare i requisiti funzionali definiti nella fase di analisi. In particolare, il sistema consente di generare report in diversi formati (*PDF*, DOC, XLSX, *HTML*) a partire da dati strutturati, garantendo flessibilità e indipendenza da piattaforme server esterne.
Dal punto di vista prestazionale, i test condotti hanno evidenziato come il comportamento del sistema sia complessivamente comparabile a quello di JasperServer, soprattutto nelle esecuzioni successive alla prima. In entrambi i casi, infatti, il tempo di inizializzazione rappresenta un fattore significativo, mentre le esecuzioni successive risultano più efficienti grazie alla riduzione dell'overhead iniziale.
Un ulteriore risultato rilevante riguarda la modularità del sistema sviluppato: la separazione tra logica di generazione, configurazione e interfaccia consente una maggiore flessibilità e facilita eventuali estensioni future. Questo aspetto rappresenta un valore aggiunto rispetto a soluzioni più monolitiche.

7.2 Limiti del lavoro

Nonostante i risultati positivi, il lavoro presenta alcuni limiti che è importante evidenziare. In primo luogo, i test prestazionali sono stati condotti in un ambiente controllato e non rappresentano pienamente scenari di produzione reale. In particolare, non sono stati eseguiti test sotto carico concorrente, che potrebbero evidenziare criticità legate alla gestione delle risorse e alla scalabilità del sistema.
In secondo luogo, il numero di scenari testati è limitato e non copre tutte le possibili casistiche applicative. Ad esempio, non sono stati considerati dataset di dimensioni molto elevate o

configurazioni distribuite.
Un ulteriore limite riguarda la gestione delle risorse del motore BIRT: l'inizializzazione e la chiusura del motore ad ogni esecuzione, seppur corretta dal punto di vista della stabilità, potrebbe risultare inefficiente in contesti ad alta frequenza di utilizzo.
Infine, l'interfaccia utente sviluppata nel *PoC* ha uno scopo puramente dimostrativo e non è stata progettata per un utilizzo in ambienti produttivi.

7.3 Sviluppi futuri

Alla luce dei risultati ottenuti, è possibile individuare diverse direzioni per sviluppi futuri del sistema.
Un primo possibile miglioramento consiste nell'ottimizzazione delle prestazioni, ad esempio attraverso l'introduzione di meccanismi di caching o la gestione persistente del motore BIRT, evitando la sua inizializzazione ad ogni esecuzione.
Un secondo sviluppo riguarda l'evoluzione dell'architettura verso un modello a servizi, ad esempio tramite la realizzazione di un'API REST che consenta di integrare il sistema in applicazioni web o microservizi.

Ulteriori miglioramenti potrebbero includere:

- supporto a nuove sorgenti dati (database relazionali, servizi esterni);
- gestione della concorrenza e ottimizzazione per scenari multi-utente;
- miglioramento dell'interfaccia utente;
- integrazione con sistemi aziendali esistenti.

Questi sviluppi consentirebbero di trasformare il Proof of Concept in una soluzione completa e utilizzabile in contesti reali.

7.4 Raggiungimento degli obiettivi

Di seguito si riporta la verifica dei requisiti individuati durante l'analisi descritta nel Capitolo 3, suddivisi in funzionali, non funzionali, qualitativi e di vincolo. Per ciascun requisito è indicato l'esito finale ottenuto al termine del progetto: soddisfatto, parzialmente soddisfatto oppure non soddisfatto.

7.4.1 Requisiti funzionali

Tabella 7.1: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Classificazione	Esito
RFO1	Il sistema deve fornire un editor grafico per consentire la creazione e la progettazione visuale dei report	Obbligatorio	Soddisfatto
RFO2	Il sistema deve permettere l'integrazione con servizi e applicazioni che espongono dati tramite *API* in formato *JSON*	Obbligatorio	Soddisfatto
RFO3	Il sistema deve poter gestire report complessi e strutturati in modo modulare, in particolare elementi di sottoreport	Obbligatorio	Soddisfatto
RFO4	Il sistema deve consentire l'esportazione dei report in molteplici formati (*PDF*, DOC, XLSX)	Obbligatorio	Soddisfatto
RFO5	Il sistema deve consentire l'invocazione programmatica dei report da applicazioni esterne tramite *API*	Obbligatorio	Soddisfatto
RFO6	Il sistema deve garantire supporto a database *SQL* e *NoSQL*	Obbligatorio	Soddisfatto
RFO7	Il sistema deve fornire la possibilità di personalizzazione e *scripting* dei report	Obbligatorio	Soddisfatto

7.4.2 Requisiti non funzionali

Tabella 7.2: Tabella del tracciamento dei requisiti non funzionali

Requisito	Descrizione	Classificazione	Esito
RNFO1	Il sistema deve supportare la scalabilità, consentendo la gestione di un numero crescente di richieste e utenti	Obbligatorio	Soddisfatto
RNFO2	Il sistema deve garantire la corretta generazione dei report in modo consistente	Obbligatorio	Soddisfatto
RNFO3	Il sistema deve consentire una facile integrazione con applicazioni esistenti sviluppate in *Java*, *.NET* e *Python*	Obbligatorio	Soddisfatto
RNFO4	Il sistema deve garantire tempi di risposta accettabili anche su dataset consistenti	Obbligatorio	Soddisfatto
RNFO5	Il sistema deve risultare mantenibile, facilitando aggiornamenti, configurazioni e interventi evolutivi	Desiderabile	Soddisfatto

7.4.3 Requisiti qualitativi

Tabella 7.3: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Classificazione	Esito
RQO1	Il sistema deve fornire un'interfaccia utente intuitiva e di facile utilizzo	Obbligatorio	Soddisfatto
RQD2	Il sistema deve garantire coerenza nella progettazione e nel *layout* dei report	Desiderabile	Soddisfatto
RQD3	Il sistema deve offrire una buona esperienza di sviluppo, supportata da documentazione chiara e *API* ben definite	Desiderabile	Soddisfatto

7.4.4 Requisiti di vincolo

Tabella 7.4: Tabella del tracciamento dei requisiti di vincolo

Requisito	Descrizione	Classificazione	Esito
RVO1	Il sistema deve poter essere installato e gestito su server locali, senza dipendere esclusivamente da soluzioni cloud	Obbligatorio	Soddisfatto
RVO2	Il sistema deve prevedere condizioni di *licensing* sostenibili, evitando vincoli restrittivi legati a licenze proprietarie	Obbligatorio	Soddisfatto
RVO3	Il sistema deve essere compatibile con l'infrastruttura e lo *stack* tecnologico aziendale esistente	Desiderabile	Soddisfatto
RVD4	Il sistema dovrebbe utilizzare tecnologie consolidate e ampiamente supportate dalla *community* o dal *vendor*	Desiderabile	Soddisfatto

7.4.5 Riepilogo dei requisiti raggiunti

Tabella 7.5: Riepilogo dei requisiti soddisfatti

Tipologia	Obbligatorio	Desiderabile	Opzionale	Totale
Funzionali	7	0	0	7
Non funzionali	4	1	0	5
Qualitativi	1	2	0	3
Vincolo	3	1	0	4
Totale	15	4	0	19

Dalla tabella illustrata si può notare che tutti i requisiti identificati durante l'analisi sono stati soddisfatti, con un totale di 19 requisiti raggiunti. In particolare, sono stati soddisfatti tutti i requisiti funzionali,non funzionali, qualitativi e di vincolo. Questo risultato conferma il successo del progetto nel raggiungere gli obiettivi prefissati e nel fornire una soluzione efficace per la generazione di report basata su BIRT.

7.5 Considerazioni finali

In conclusione, il lavoro svolto ha dimostrato la fattibilità della realizzazione di un sistema di reportistica basato su BIRT completamente personalizzato e indipendente da piattaforme server esterne.
Il Proof of Concept sviluppato rappresenta una valida alternativa a soluzioni consolidate, almeno per quanto riguarda scenari di utilizzo standard, offrendo al contempo maggiore flessibilità e controllo sull'intero processo di generazione dei report.
L'esperienza maturata durante lo sviluppo ha inoltre permesso di approfondire tematiche rilevanti nell'ambito dell'ingegneria del software, quali la progettazione architetturale, l'integrazione tra componenti e l'analisi delle prestazioni.
Nel complesso, i risultati ottenuti confermano il raggiungimento degli obiettivi prefissati e forniscono una base solida per eventuali evoluzioni future del sistema.

7.6 Valutazione personale

Il progetto di stage ha rappresentato un'importante opportunità di crescita professionale e personale, consentendomi di applicare le conoscenze acquisite durante il percorso universitario a un contesto reale e complesso.
Ringrazio l'azienda per avermi dato la possibilità di lavorare su un progetto così stimolante e per il supporto ricevuto durante tutto il periodo di stage.
Ho avuto modo di confrontarmi con sfide tecniche significative, sviluppando competenze specifiche nell'ambito della generazione di report e dell'integrazione di sistemi software, nonché migliorando le mie capacità di problem solving, gestione del tempo e lavoro in team.
Con questa esperienza poi ho potuto migliorare le mie capacità di analisi, progettazione e sviluppo software, permettendomi di riuscire a conciliare attività di ricerca e sviluppo con le esigenze di un progetto concreto, rispettando scadenze e obiettivi prefissati.
Inoltre, l'esperienza mi ha permesso di capire meglio quale anmbito dell'informatica sia più adatto alle mie competenze e alla mia persona.
Ci sono statti anche dei limiti e delle difficoltà, specie all'inizio del progetto, quando ho dovuto affrontare una curva di apprendimento piuttosto ripida per comprendere le tecnologie e gli strumenti utilizzati. Tuttavia, grazie al supporto del team e alla mia determinazione, sono riuscito a superare queste difficoltà e a portare a termine con successo il progetto.
Nel complesso sono soddisfatto dei risultati ottenuti e grato per l'opportunità di aver potuto contribuire a questo progetto di ricerca.

Acronimi e abbreviazioni

API Application Program Interface. 48

Docker Docker. 48

HTML Hypertext Markup Language. 48

IDE Integrated Development Environment. 48

Java Java. 48

JSON JavaScript Object Notation. 48

JVM Java Virtual Machine. 49

PDF Portable Document Format. 49

PoC PoC. 49

Python Python. 49

REST Representational State Transfer API. 49

SQL Structured Query Language. 49

Glossario

API in informatica con il termine *Application Programming Interface API* (ing. interfaccia di programmazione di un'applicazione) si indica ogni insieme di procedure disponibili al programmatore, di solito raggruppate a formare un set di strumenti specifici per l'espletamento di un determinato compito all'interno di un certo programma. La finalità è ottenere un'astrazione, di solito tra l'hardware e il programmatore o tra software a basso e quello ad alto livello semplificando così il lavoro di programmazione. 4, 7, 8, 10, 12–17, 22, 23, 27, 29, 30, 32–36, 44, 45, 47

Docker *Docker* è una piattaforma software che consente di creare, distribuire e gestire applicazioni in container. Un container è un'unità leggera e portatile che include tutto il necessario per eseguire un'applicazione, come il codice, le librerie, le dipendenze e le configurazioni. Docker semplifica il processo di sviluppo, test e distribuzione delle applicazioni, consentendo agli sviluppatori di creare ambienti isolati e coerenti su qualsiasi sistema operativo o infrastruttura. Docker è ampiamente utilizzato per la gestione di microservizi, l'orchestrazione di container e la creazione di ambienti di sviluppo replicabili. 17, 19, 47

HTML *HTML, Hypertext Markup Language* (ing. linguaggio di marcatura ipertestuale) è un linguaggio di markup utilizzato per creare e strutturare contenuti su pagine web. HTML consente agli sviluppatori di definire la struttura e il layout di una pagina web, nonché di incorporare testo, immagini, video e altri elementi multimediali. HTML è uno dei pilastri fondamentali del World Wide Web ed è supportato da tutti i browser web moderni. 10, 11, 13–16, 23, 25, 28, 30–36, 38, 42, 47

IDE *IDE, Integrated Development Environment* (ing. ambiente di sviluppo integrato) è un software che fornisce agli sviluppatori un insieme completo di strumenti per la scrittura, il debug e la gestione del codice sorgente. Un IDE tipicamente include un editor di codice, un compilatore o interprete, un debugger e strumenti per la gestione dei progetti. L'obiettivo di un IDE è semplificare il processo di sviluppo software, migliorare la produttività e facilitare la gestione del ciclo di vita del software. 18, 22, 47

Java *Java* è un linguaggio di programmazione a oggetti, open source e platform-independent. È uno dei linguaggi di programmazione più popolari al mondo, ampiamente utilizzato per lo sviluppo di applicazioni web, mobili e desktop. 3, 4, 7, 10, 11, 13, 22, 24, 25, 28–31, 33–36, 38, 44, 47

JSON *JSON, JavaScript Object Notation* (ing. linguaggio di notazione di oggetto JavaScript) è un formato di dati leggero e facile da leggere e scrivere. JSON è ampiamente utilizzato per lo scambio di dati tra client e server in applicazioni web, nonché per la memorizzazione di dati strutturati in file o database. JSON rappresenta i dati come coppie chiave-valore, consentendo una facile serializzazione e deserializzazione dei dati

in vari linguaggi di programmazione. 4, 7, 15–17, 21, 25, 26, 28, 30, 32, 36–38, 40–42, 44, 47

JVM *JVM, Java Virtual Machine* (ing. macchina virtuale Java) è un'implementazione software di una macchina virtuale che esegue programmi scritti in Java. La JVM fornisce un ambiente di esecuzione indipendente dalla piattaforma, consentendo ai programmi Java di essere eseguiti su qualsiasi sistema operativo o hardware che supporti la JVM. La JVM gestisce la memoria, l'esecuzione del codice e altre funzionalità necessarie per eseguire i programmi Java in modo efficiente e sicuro. 22, 47

PDF *PDF, Portable Document Format* (ing. formato di documento portatile) è un formato di file sviluppato da Adobe Systems per rappresentare documenti in modo indipendente dall'hardware, dal software e dal sistema operativo utilizzati per crearli o visualizzarli. I file PDF possono contenere testo, immagini, grafica vettoriale e altri elementi multimediali, mantenendo la formattazione originale del documento. Il formato PDF è ampiamente utilizzato per la condivisione di documenti elettronici, in quanto garantisce che il contenuto venga visualizzato correttamente su diverse piattaforme e dispositivi. 7, 10, 11, 13, 14, 16, 25, 26, 30, 32, 36, 42, 44, 47

PoC *PoC, Proof of Concept* (ing. prova di concetto) è una dimostrazione pratica o un prototipo sviluppato per verificare la fattibilità di un'idea, di un concetto o di una tecnologia. Un PoC viene spesso utilizzato per testare l'efficacia di una soluzione proposta prima di investire risorse significative nello sviluppo completo. Il PoC può essere utilizzato in vari contesti, come lo sviluppo software, la sicurezza informatica, l'ingegneria e altri campi, al fine di valutare se un'idea o una tecnologia può essere implementata con successo e soddisfare i requisiti desiderati. 5, 20, 21, 28, 29, 31, 37–43, 47

Python *Python* è un linguaggio di programmazione a oggetti, open source e platform-independent. È uno dei linguaggi di programmazione più popolari al mondo, ampiamente utilizzato per lo sviluppo di applicazioni web, mobili e desktop. 3, 4, 7, 22, 23, 28–32, 34–36, 44, 47

REST *REST, Representational State Transfer API* (ing. trasferimento di stato rappresentazionale) è uno stile architetturale per la progettazione di servizi web. REST si basa su un insieme di principi e vincoli che guidano la progettazione e l'implementazione di API (Application Programming Interface) che consentono la comunicazione tra client e server. Le API RESTful utilizzano i metodi HTTP standard (come GET, POST, PUT, DELETE) per eseguire operazioni sui dati e sono progettate per essere semplici, scalabili e facilmente integrabili con altre applicazioni. 3, 4, 21, 23, 25, 33, 47

SQL *SQL, Structured Query Language* (ing. linguaggio strutturato di interrogazione) è un linguaggio standardizzato per la gestione e l'interrogazione dei database relazionali. È utilizzato per definire e manipolare i dati in un database, consentendo operazioni come la creazione, l'aggiornamento, la cancellazione e il recupero di informazioni. SQL è ampiamente utilizzato in applicazioni software e sistemi di gestione di database per interagire con i dati in modo efficiente e strutturato. 7, 15, 44, 47

Bibliografia

Siti web consultati

BIRT - Business Intelligence and Reporting Tools. URL: https://eclipse-birt.github.io/birt-website/ (cit. a p. 9).

Docker. URL: https://www.docker.com/ (cit. a p. 14).

DynamicPDF. URL: https://www.dynamicpdf.com/ (cit. a p. 10).

FineReport. URL: https://www.finereport.com/ (cit. a p. 10).

Helical Insight. URL: https://www.helicalinsight.com/ (cit. a p. 11).

JasperReports Library. URL: https://community.jaspersoft.com/project/jasperreports-library (cit. a p. 9).

Knowage. URL: https://www.knowage-suite.com/ (cit. a p. 12).

Pentaho. URL: https://pentaho.com/ (cit. a p. 12).

Postman - API Development Environment. URL: https://www.postman.com/ (cit. a p. 15).

Report Server. URL: https://reportserver.net/en/ (cit. a p. 13).

SAP Crystal Reports. URL: https://www.sap.com/products/crystal-reports.html (cit. a p. 13).

Stimulsoft. URL: https://www.stimulsoft.com/en/ (cit. a p. 14).

Visual Studio Code. URL: https://code.visualstudio.com/ (cit. a p. 23).

VisualVM - Visual Tool for Java. URL: https://visualvm.github.io/ (cit. a p. 38).